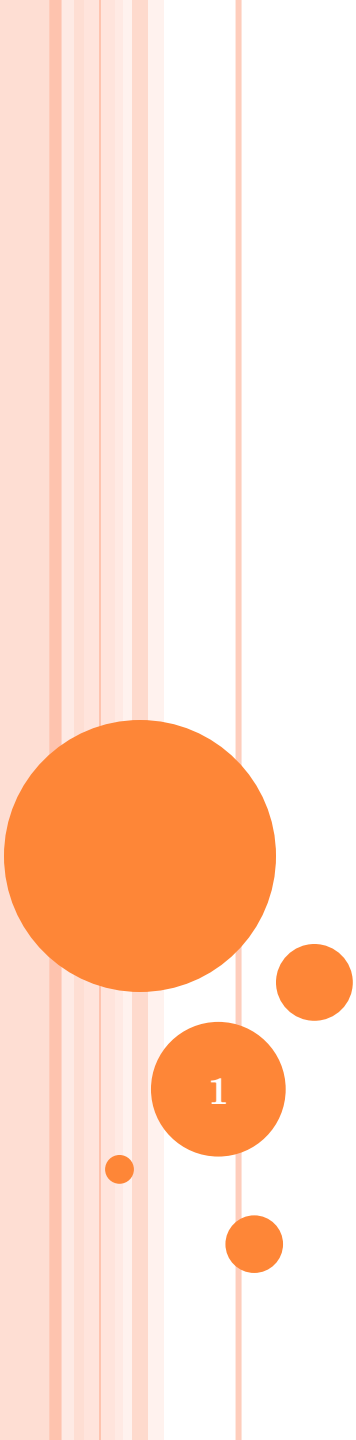




SISTEME DE OPERARE

Curs 10: Gestiunea memoriei (partea 1)

Kinga Márton

Universitatea Tehnică din Cluj-Napoca
Departamentul de Calculatoare

GESTIUNEA MEMORIEI

- În cuprins

- **Concepte generale** legate de gestiunea memoriei

- Virtualizarea memoriei
- Spațiu de adrese
- Relocare, protecție și partajare
- Legarea adreselor
- Swapping

- **Mecanisme** de gestiune a memoriei

- Partiționare
- Paginare
- Segmentare

MEMORIA

- Un **tablou** de dimensiuni foarte mari
 - Fiecare cuvânt – sau byte are propria adresă (index)
- Componentă fundamentală a sistemului de calcul
 - Programul trebuie **încărcat în memorie** pentru a putea fi rulat în cadrul unui proces
 - Procesorul **încarcă instrucțiunile din memorie** în funcție de valoarea PC (Program Counter)
 - Instrucțiunea este decodificată și poate provoca **aducerea unor operanzi din memorie** (date)
 - **Rezultatul** efectuării unei instrucțiuni poate fi **stocat în memorie**
- Unitatea de memorie vede doar un **șir de adrese** de memorie
 - Nu știe cum sunt generate aceste adrese și ce anume este stocat la aceea adresă (instrucțiune sau dată)
- Gestiunea memoriei este **sarcina SO**

VIRTUALIZAREA MEMORIEI – MOTIVAȚIE

- Procesele și threadurile partajează resursele sistemului
 - **Procesorul** (am văzut în cursurile precedente)
 - Necesitatea multiplexării procesorului (în timp)
 - Sisteme cu multiprogramare și time-sharing
 - PRIN PLANIFICAREA EXECUȚIEI SE ÎMBUNĂTĂȚEȘTE UTILIZAREA PROCESORULUI ȘI TIMPUL DE RĂSPUNS
 - Condiție: existența mai multor procese încărcate în memorie
 - **Memoria** (începem studiul azi)
 - Necesitatea multiplexării memoriei (în spațiu)
 - Studiem diferite metode de gestiune a memoriei
 - DE LA CELE MAI SIMPLE PÂNĂ LA PAGINARE ȘI SEGMENTARE
 - Fiecare are avantaje și dezavantaje specifice
 - Alegerea unui algoritm depinde de mai mulți factori
 - ÎN SPECIAL DE HARDWARE-UL SISTEMULUI
 - Majoritatea metodelor necesită suport hardware

GESTIUNEA MEMORIEI

- Dacă integrăm unui volum foarte mare de memorie
-> Mai este necesară gestiunea?

DA

- **Legea lui Parkinson**

- Versiunea originală
 - “Work expands so as to fill the time available for its completion”
- Aplicată în domeniul CS
 - “Data expands to fill the space available for storage”

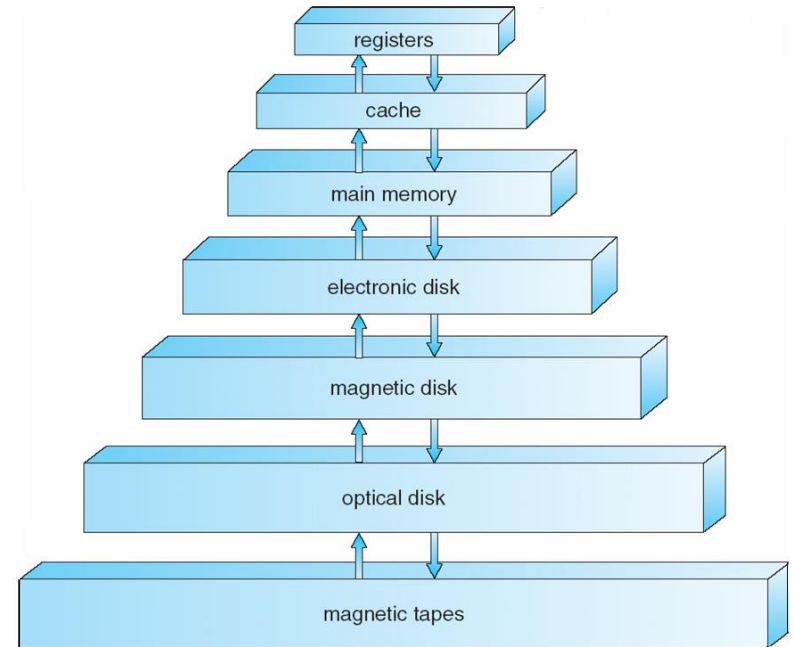
sau

- “Programs expand to fill the memory available to hold them”

Sună cunoscut? :-)

MEMORIA - CE ȘTIM PÂNĂ ACUM

- Resursă importantă & costisitoare
- Caz ideal: ne dorim
 - Memorie rapidă, nevolatilă, mare și ieftină
 - Dar tehnologia nu furnizează astfel de memorie
- Caz real: compromis între viteză, cost, capacitate
- **Ierarhia de memorie**
 - Cantitate mică de memorie foarte rapidă, foarte costisitoare și volatilă
 - Cantitate relativ mică de memorie de viteză medie, cost mediu și volatilă
 - Cantitate semnificativă de memorie nevolatilă, ieftină și de viteză mică



Silberschatz, pg. 11

	Capacity	Latency	Cost/GB
Register	1000s of bits	20 ps	\$\$\$\$
SRAM	~10 KB-10 MB	1-10 ns	~\$1000
DRAM	~10 GB	80 ns	~\$10
Flash*	~100 GB	100 us	~\$1
Hard disk*	~1 TB	10 ms	~\$0.10

Processor
Datapath
Memory
Hierarchy
I/O
subsystem

MIT, Memory Technologies

* non-volatile (retains contents when powered off)

IERARHIA DE MEMORIE – VALORI TIPICE

- Fiecare nivel poate fi văzut ca un cache pentru nivelul de mai jos

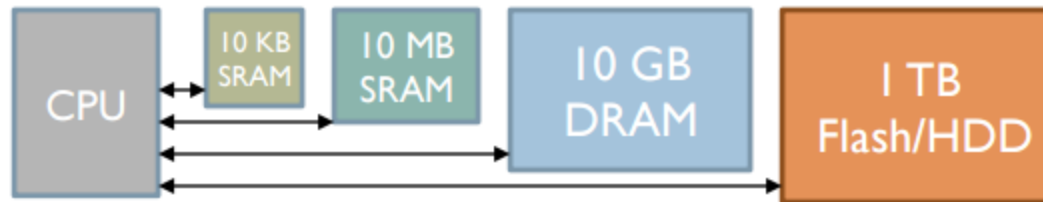
		Access time	Capacity	Managed By
On the datapath	Registers	1 cycle	1 KB	Software/Compiler
	↕			
	Level 1 Cache	2-4 cycles	32 KB	Hardware
	↕			
	Level 2 Cache	10 cycles	256 KB	Hardware
	↕			
On chip	Level 3 Cache	40 cycles	10 MB	Hardware
	↕			
Other chips	Main Memory	200 cycles	10 GB	Software/OS
	↕			
	Flash Drive	10-100us	100 GB	Software/OS
	↕			
Mechanical devices	Hard Disk	10ms	1 TB	Software/OS

IERARHIA DE MEMORIE

- Două abordări în furnizarea interfeței

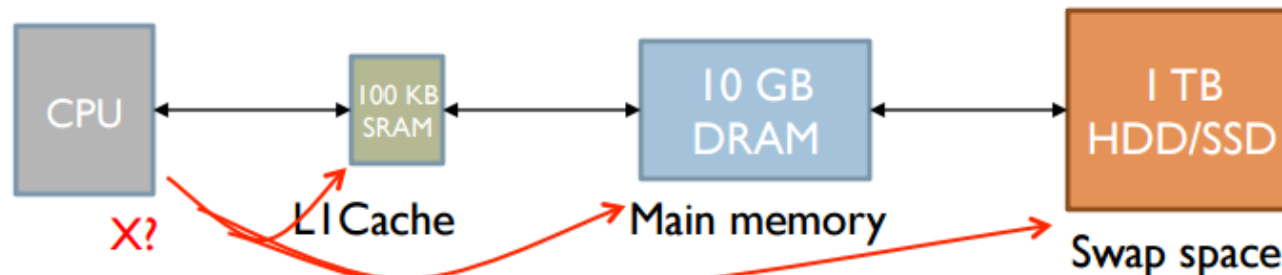
1. Expunerea ierarhiei

- Utilizarea eficientă este responsabilitatea programatorului



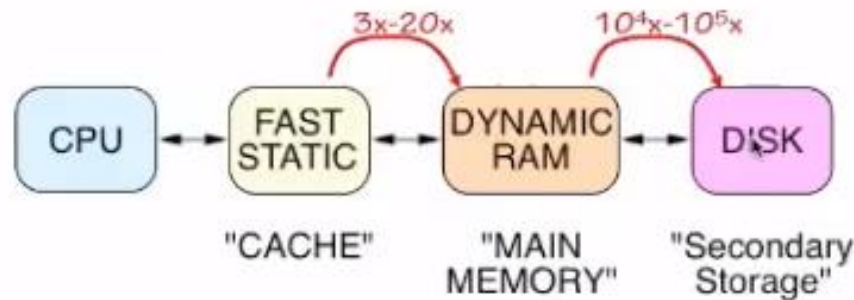
2. Ascunderea ierarhiei

- Utilizarea eficientă este responsabilitatea sistemului
 - Pe baza șabloanelor de utilizare (localitatea referințelor)



IERARHIA DE MEMORIE

- Folosirea în mod transparent a mai multor niveluri de memorie
 - Memorie foarte rapidă dar de dimensiuni reduse + memorie mai mare dar mai lentă => iluzia unei memorii mari și rapide



- Fiecare nivel poate fi văzut ca un cache pentru nivelul de mai jos
- Memoria virtuală
 - Extensie a memoriei interne cu memorie pe disc
 - RAM-ul are rolul de cache pt dispozitivele de stocare mai mari, dar mult mai lente

MEMORIA CACHE

MEMORIA CACHE

- Componentă de memorie de dimensiune redusă plasată între procesor și memoria internă
 - Reține în mod transparent **datele accesate recent**
 - Principiul localității
 - Acces rapid la datele aflate în cache
 - Acces mai lent la date care nu sunt în cache și trebuie accesate din memoria internă
- Se folosesc mai multe niveluri de cache
 - În general nivelurile L1 și L2 sunt proprii fiecărui core și L3 (și L4) este partajat

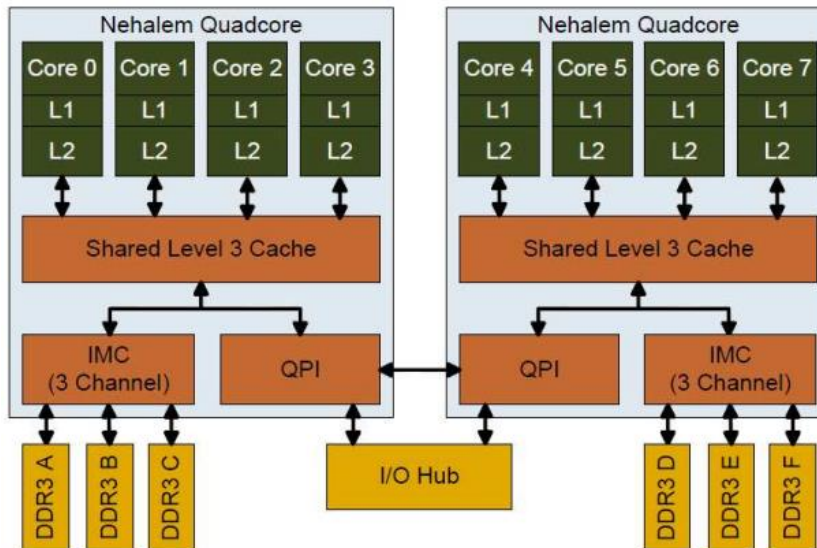
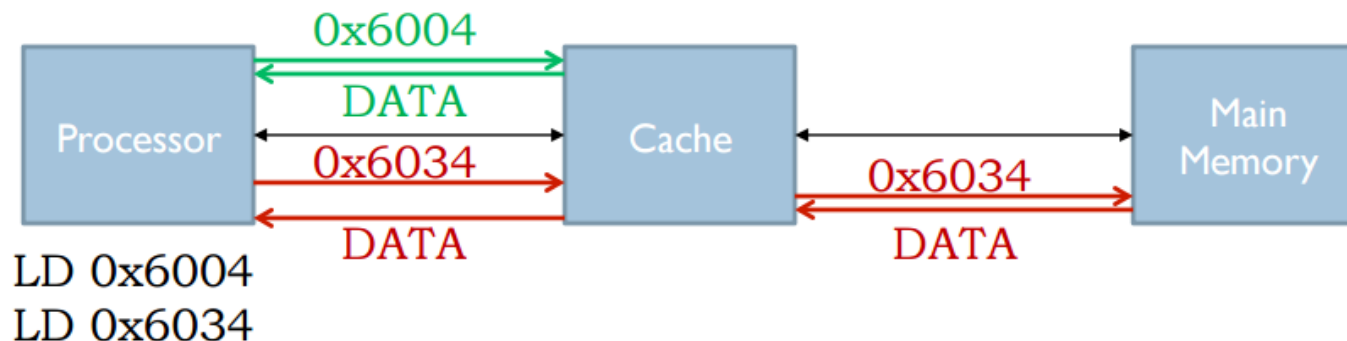


Fig. 1. Eight-core Nehalem Processor

MEMORIA CACHE – ACCESUL LA DATE

- Procesorul transmite adresa (pe care dorește să o acceseze) cache-ului
- 2 opțiuni
 - **Cache hit**
 - Data de la adresa cerută este în cache
 - Se returnează rapid procesorului
 - **Cache miss**
 - Data cerută nu este în cache
 - Data trebuie adusă din memoria internă și trimisă procesorului
 - Data este salvată în cache (posibil prin suprascrierea altei locații)
- => Timpul de acces este variabil



MEMORIA CACHE - METRICI

- Timpul mediu de acces la memorie
 - Numit: Average Memory Access Time (AMAT)

Hit Ratio: $HR = \frac{hits}{hits + misses} = 1 - MR$

Miss Ratio: $MR = \frac{misses}{hits + misses} = 1 - HR$

Average Memory Access Time (AMAT):

$$AMAT = HitTime + MissRatio \times MissPenalty$$

- Scopul memoriei cache este îmbunătățirea AMAT
- Formula poate fi aplicată recursiv pe multiplele niveluri ale memoriei cache

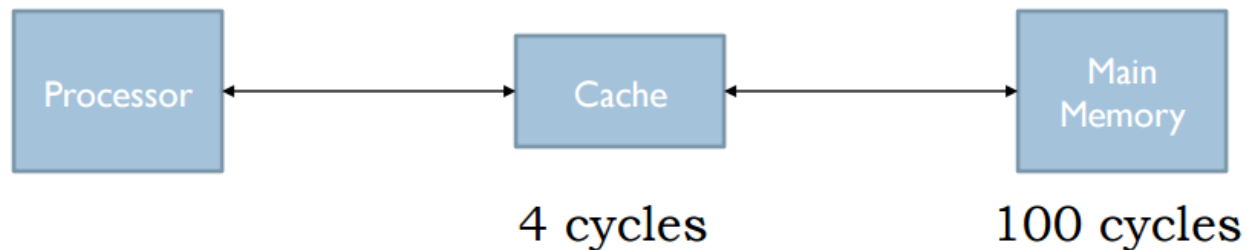
$$AMAT = HitTime_{L1} + MissRatio_{L1} \times AMAT_{L2} =$$

$$AMAT = HitTime_{L1} + MissRatio_{L1} \times (HitTime_{L2} + MissRatio_{L2} \times AMAT_{L3}) = \dots$$

MEMORIA CACHE – EXERCİȚIU AMAT

- Considerăm situația

- Regăsirea datei existente în cache durează 4 cicluri de ceas
- Regăsirea datei care nu este în cache – trebuie adusă din memoria internă - durează 100 cicluri de ceas



- Ce valoare de Hit Ratio (rată de regăsire) ar trebui să avem dacă dorim să atingem $AMAT = 100$ cicluri de ceas?
 - Similar cu situația în care avem doar memoria internă - acces în 100 de cicluri de ceas

$$100 = 4 + (1 - HR) \times 100 \Rightarrow HR = 4\%$$

- Ce valoare de Hit Ratio (rată de regăsire) ar trebui să avem dacă dorim să atingem $AMAT = 5$ cicluri de ceas?

$$5 = 4 + (1 - HR) \times 100 \Rightarrow HR = 99\%$$

MEMORIA CACHE – POLITICA DE SCRIERE

- Definește cum sunt **sincronizate datele dintre cache și memoria internă** când CPU-ul modifică date

- influențează:
 - performanța
 - consistența datelor
 - traficul către RAM

Dacă valoarea este deja în cache:

- se modifică doar cache-ul?
- se modifică și RAM-ul imediat?
- RAM-ul se actualizează mai târziu?

- **Write-through**

- fiecare scriere făcută în cache este copiată imediat și în memoria RAM pentru a menține datele sincronizate permanent
 - Simplu, sigur, lent

- **Write-back**

- scrierile sunt făcute inițial doar în cache, iar RAM-ul este actualizat mai târziu, când linia de cache este înlocuită
 - Rapid, eficient, complex
 - Programele modifică des aceleași date

- Majoritatea cache-urilor moderne L1/L2/L3 folosesc predominant **write-back** pentru performanță ridicată

GESTIUNEA MEMORIEI INTERNE

GESTIUNEA MEMORIEI INTERNE

- Gestiunea ierarhiei de dispozitive de memorare
 - Virtualizarea memoriei
 - Principala sarcină a **managerului de memorie**
 - **Alocarea** memoriei pentru procese
 - **Recuperarea** memoriei eliberate
 - **Contabilizarea** memoriei utilizate
 - Gestiunea mecanismelor de **swapping**
- Proprietăți urmărite
 - Transparență – față de programul în execuție
 - Eficiență
 - În timp – să nu introducă întârzieri notabile în rularea programelor
 - În spațiu – să nu folosească prea mult spațiu de stocare pentru gestiunea structurilor de date necesare virtualizării
 - Protecție - între procese și SO și între diferite procese
 - Proprietatea de izolare între procese
 - Siguranță față de interferențe negative - intenționate sau involuntare

MEMORIA

○ Memoria internă și regiștrii procesorului

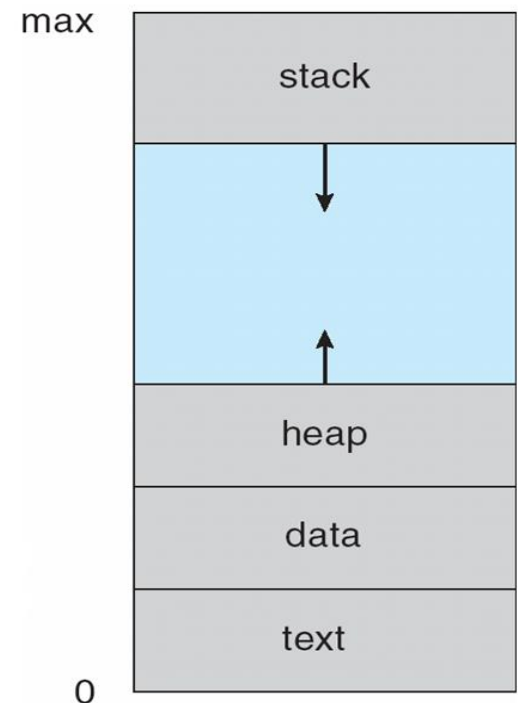
- singurele sisteme de stocare care pot fi **accesate direct de către procesor**
 - Instrucțiunile mașină nu pot avea argumente adrese de pe disc
 - Datele necesare instrucțiunilor trebuie încărcate în memorie pentru a fi accesate
- Regiștrii integrați în procesor
 - sunt accesibili într-un singur tact al ceasului intern (sau chiar mai rapid)
- Memoria internă
 - este accesibilă prin magistrala de memorie
 - Poate dura mai multe cicluri de ceas
 - Soluție – interpunerea între procesor și memoria internă a unui buffer de viteză mare – **memoria cache**

○ Trebuie asigurată corectitudinea operațiilor

- Protejarea SO de alte procese și a proceselor între ele
 - Introduce necesitatea **spațiilor de adrese** separate (“cocon” propriu)

SPAȚIUL DE ADRESE

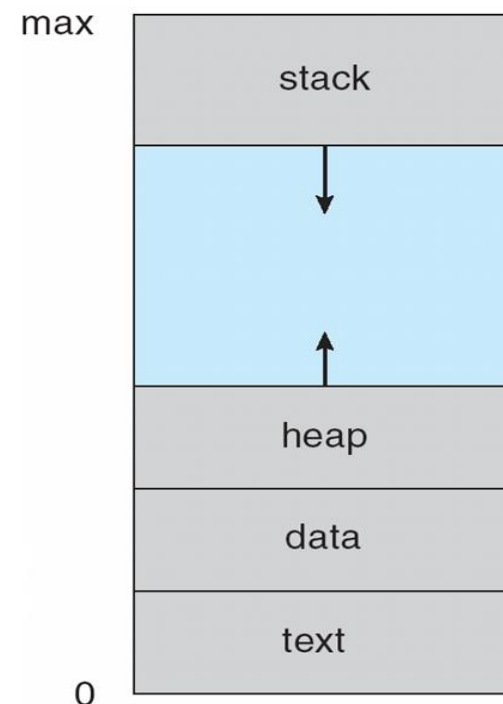
- Viziunea programului în execuție despre memoria sistemului
 - Abstracțiunea memoriei fizice
 - Concept fundamental pt înțelegerea modului de virtualizare a memoriei
- Spațiul de adrese al unui proces conține
 - Segmentul de cod (text)
 - Segmentul de date
 - Heap
 - Stiva



Silberschatz, pg. 102

SPAȚIUL DE ADRESE

- Spațiul de adrese al unui proces
 - **Segmentul de cod (text)**
 - Codul programului - instrucțiunile în cod mașină
 - De regulă este read-only (deci nu crește și nu scade)
 - **Segmentul de date**
 - Datele programului
 - Variabile globale, statice, constantele, etc.
 - 2 părți
 - Date inițializate
 - Variabile și constante de compilare care necesită o valoare inițială la pornire
 - Dp dv al SO această parte seamănă cu segmentul de cod
 - Ambele trebuie încărcate în memorie la pornirea programului
 - Date neinițializate (BSS)
 - Optimizare
 - Variabile globale și statice neinițializate
 - Implicit inițializate la 0
 - **Heap**
 - memorie alocată dinamic de către utilizator (malloc)
 - Se poate modifica – dimensiunea crește și scade în funcție de alocarea / dealocare memoriei
 - **Stiva**
 - Începe de la adrese mari (capătul spațiului de adrese) și crește către adrese mici
 - Stochează variabile locale, parametri formali, valori de retur, adrese de retur pentru funcții

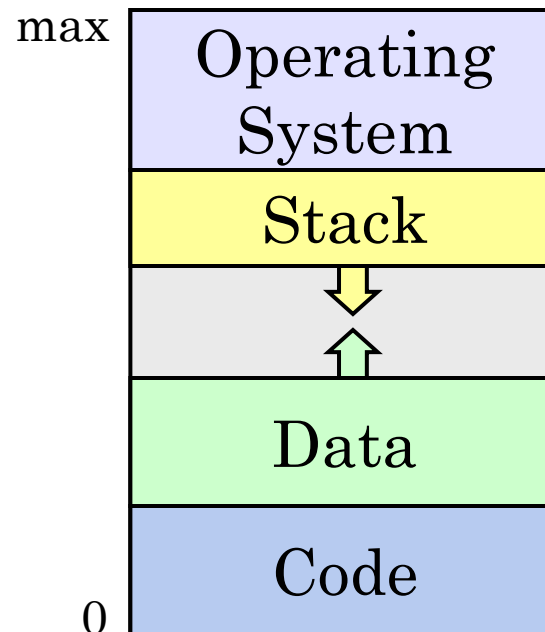


Silberschatz, pg. 102

SPAȚIUL DE ADRESE

○ Primele sisteme

- Nu furnizau o viziune abstractă asupra gestiunii memoriei
- SO – set de rutine (o bibliotecă de fapt) în memorie
- Un singur program în execuție (un proces) încărcat în memoria fizică și care folosește tot restul memoriei
 - nu era nevoie de crearea unei iluzii – procesul chiar avea aproape toată memoria la dispoziție



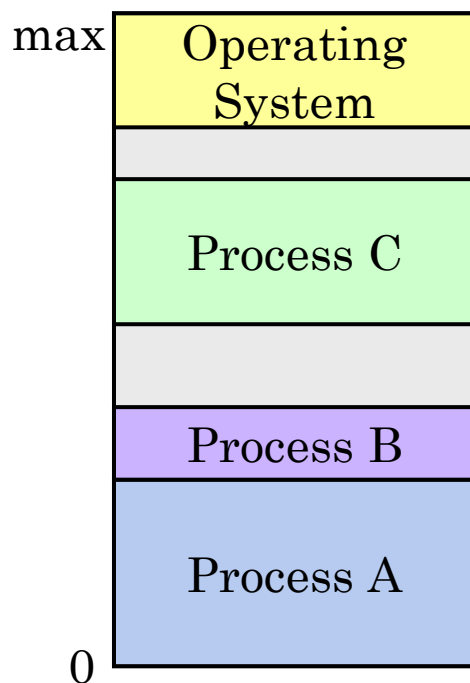
SPAȚIUL DE ADRESE

○ Multiprogramare

- Mai multe procese încărcate în memorie
 - SO comută între ele (de ex. când procesul activ apelează I/O)
 - Utilizare mai eficientă a procesorului

○ Partajarea timpului (time sharing)

- Importanța sistemelor interactive – accent pe timpul de răspuns
- Mai multe procese încărcate în memorie
 - SO comută între ele la expirarea timpului alocat (time slice) sau I/O



○ Trei procese în memorie

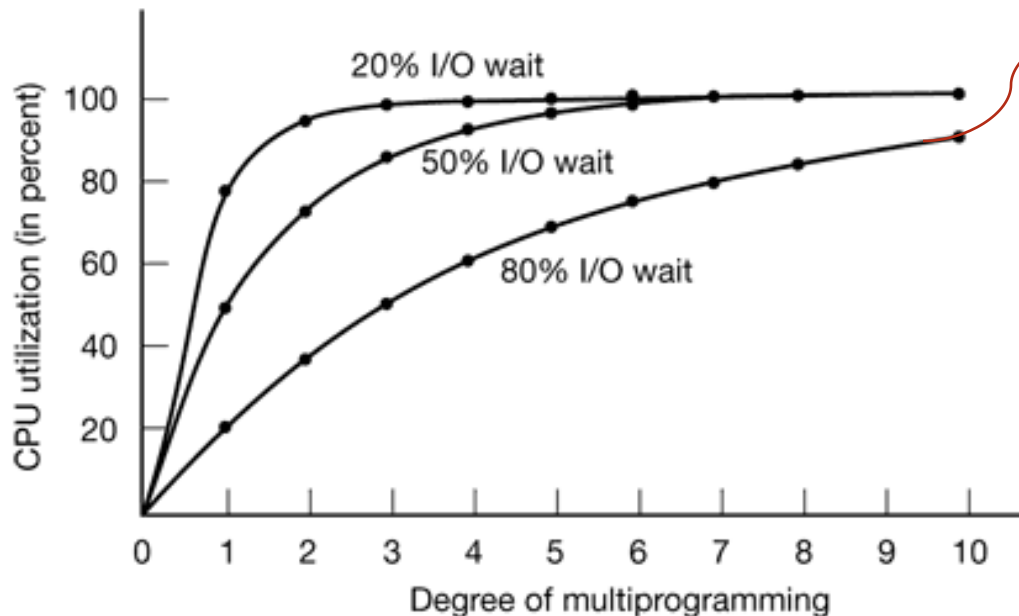
- A, B, C
 - Unul în execuție și celelalte în coada de așteptare (Ready, etc.)

○ Apar probleme

- Protecție
- Relocare
- Partajare

MAI MULTE PROCESE ÎN MEM. – MOTIVAȚIE

- Model probabilistic - aproximare simplistă, dar utilă
 - Avem n procese în memorie
 - n – gradul de multiprogramare
 - Fiecare proces petrece o fracțiune p din timp așteptând pentru i/e
 - p^n probabilitatea ca toate procesele să aștepte simultan pentru i/e
 - $1 - p^n$ utilizarea procesorului



Dacă procesele așteaptă 80% din timp pt i/e atunci trebuie să existe cel puțin 10 procese în memorie pentru ca pierderea în utilizarea CPU să fie sub 10%

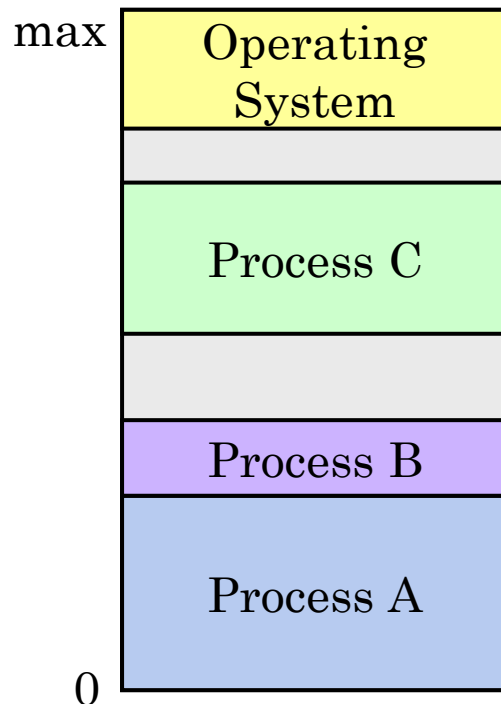
SPAȚIUL DE ADRESE

○ Multiprogramare

- Mai multe procese încărcate în memorie
 - SO comută între ele (de ex. când procesul activ apelează I/O)
 - Utilizare mai eficientă a procesorului

○ Partajarea timpului (time sharing)

- Importanța sistemelor interactive – accent pe timpul de răspuns
- Mai multe procese încărcate în memorie
 - SO comută între ele la expirarea timpului alocat (time slice)



○ Trei procese în memorie

- A, B, C
 - Unul în execuție și celelalte în coada Ready de așteptare (Ready, etc.)

○ Apar probleme

○ protecția

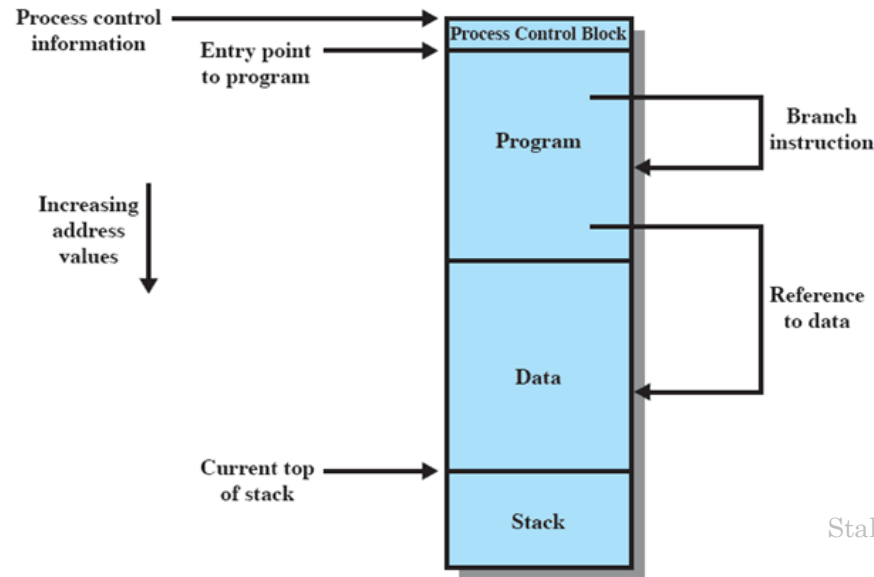
- Procesele să nu citească și să nu scrie memoria altor procese sau a SO

○ relocarea

- Adresa la care se încarcă procesul în memorie poate varia – deci nu putem folosi adrese absolute

RELOCARE

- Nu cunoaștem dinainte **locația unde va fi încărcat procesul**
 - În momentul scrierii codului nu putem ști ce alte procese vor mai fi active (încărcate în memorie) la momentul rulării programului
- Memoria internă poate fi **insuficientă** pt toate procesele
 - Procesele active trebuie să poată fi swapate (depozitate temporar pe disc)
 - Procesul activ este transferat pe disc și un alt proces este încărcat în memorie
 - La readucere în memorie procesul poate fi relocat într-o altă zonă – la o altă adresă
 - Apar probleme legate de adresare – în cadrul programului
- Deci: **nu putem folosi adrese absolute!**



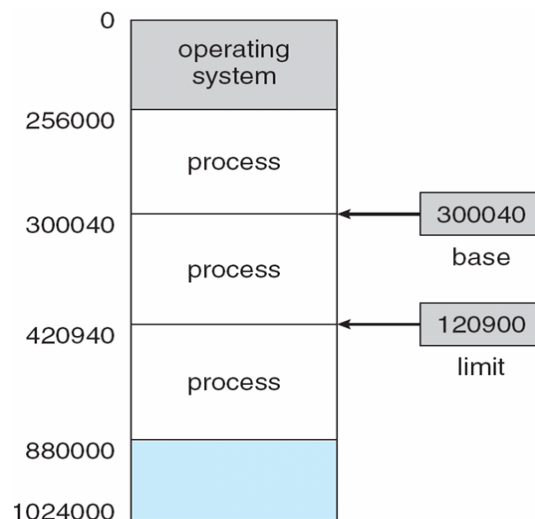
PROTECȚIE

○ Protecție

- Asigurarea **accesului exclusiv** la spațiul de adrese al procesului
 - Determinarea domeniului de adrese legale pe care procesul le poate accesa
 - Asigurarea că procesul accesează doar acest domeniu

• Soluție

- Folosirea a doi regiștrii care indică începutul și dimensiunea spațiului
 - **Base**: cea mai mică adresă fizică accesabilă
 - **Limit**: dimensiunea domeniului

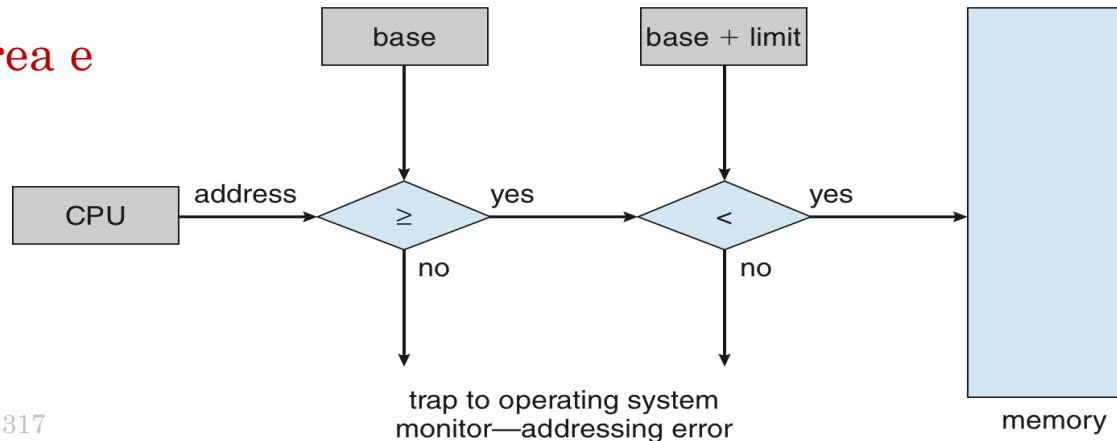


PROTECȚIE

○ Base și limit

- Pereche de regiștrii de pe procesor – valorile specifice procesului trebuie încărcate din BCP înainte de rulare
- Creează iluzia rulării pe o mașină dedicată, cu memoria începând de la adresa 0
- Toate adresele generate sunt comparate cu acești regiștrii – în timpul rulării
 - Mecanism hw
- Accesul la perechea de regiștrii se poate face doar din mod privilegiat (kernel)
- Protecția este oferită de procesor (hw) în timpul rulării, nu de SO

Deci partajarea e imposibilă?



PARTAJARE

- Flexibilitate în mecanismele de protecție
 - Permite mai multor procese să acceseze aceeași porțiune a memoriei interne
 - Ex. mai multe procese execută același program – codul este identic
 - Procese cooperante – trebuie să acceseze aceleași structuri de date
- Gestiunea memoriei trebuie să permită acces controlat la zone partajate fără a compromite protecția
- **Veste bună:** mecanismele care oferă relocare oferă și suport pentru protecție și partajare

LEGAREA ADRESELOR

LEGAREA ADRESELOR

- Adresele din cadrul programului sunt **simbolice**
 - Ex. folosim numele unei variabile pentru a ne referi la o locație de memorie
- Compilatorul leagă adresele simbolice la **adrese relocabile**
 - Ex. la 20 de octeți de la începutul acestui modul
- Editorul de legături - linker (sau încărcătorul - loader) leagă adresele relocabile la **adrese absolute**
 - Ex. 0xFFBC
- Fiecare legare este o **mapare** dintr-un spațiu de adrese în altul

LEGAREA ADRESELOR

○ Legarea se poate efectua

○ La **compilare**

- Compilatorul generează adrese fizice
 - Trebuie să cunoască dinainte unde va fi încărcat procesul
- Dacă locația se schimbă -> programul trebuie recompilat

○ La **linkeditare (static linking)**

- Compilatorul generează adrese relocabile și referă adrese externe
- Linkeditorul convertește adresele relocabile în adrese fizice și rezolvă referințele externe
 - Trebuie să cunoască dinainte unde va fi încărcat procesul
- Aplicația poate fi încărcată doar la adresa specificată și nu poate fi relocată

○ La **încărcare**

- Similar cu legarea la linkeditare, dar locația de încărcare nu este fixă
- Cod relocabil – poate fi încărcat oriunde
- Programul poate fi relocat dar numai în întregime (nu poate fi rupt în bucăți – unele încărcate, altele nu)

○ La **execuție**

- Traducere dinamică în timpul execuției
- Procesul poate fi relocat în timpul execuției dintr-un segment în altul
- Necesită suport hardware (ex. regiștrii base și limit)

ADRESE VIRTUALE VS. ADRESE FIZICE

- Conceptul de **spațiu de adrese virtual** (logic) mapat la un **spațiu de adrese fizic** este fundamental pentru gestiunea memoriei

- **Adresă logică / virtuală**

- Adrese cu care lucrează procesorul
 - Generate de program
 - Relative la spațiul de adrese virtual

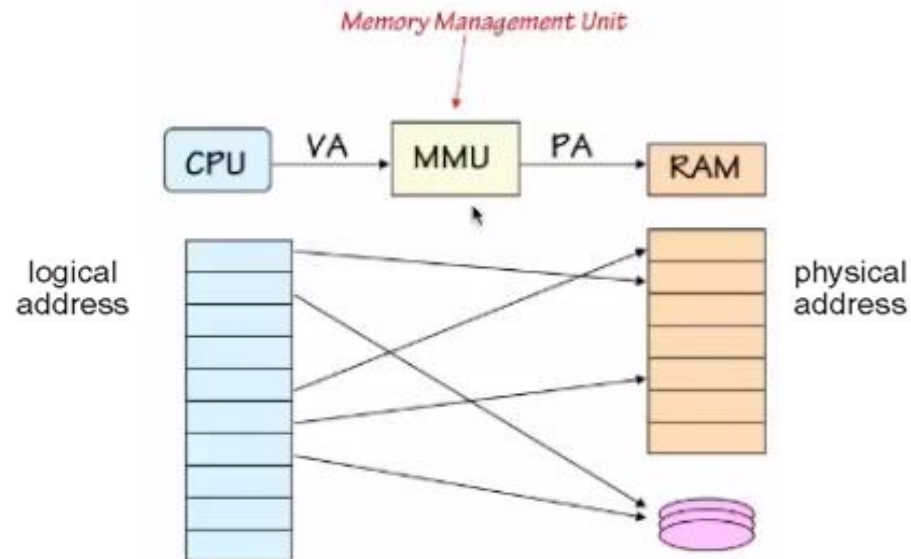
- **Adresă fizică**

- Adresele cu care lucrează unitatea de memorie
 - Adresa locației din memoria internă fizică

- Programele utilizator lucrează doar cu adrese virtuale
 - Nu văd adresele fizice
 - **Traducerea adresei** - legarea la adresa fizică - se face în momentul referențierii unei locații de memorie – cu ajutorul hardware-ului (pentru eficiență)

MMU – MEMORY-MANAGEMENT UNIT

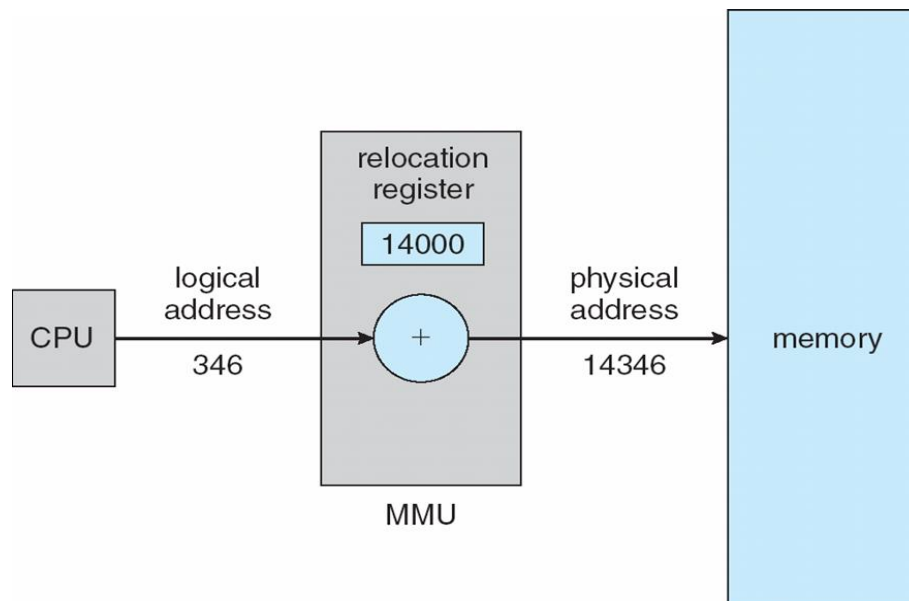
- Unitatea de gestiune a memoriei
 - componentă a procesorului
 - Mecanism HW pentru maparea adreselor virtuale la adrese fizice



MMU – MEMORY-MANAGEMENT UNIT

○ Relocare dinamică

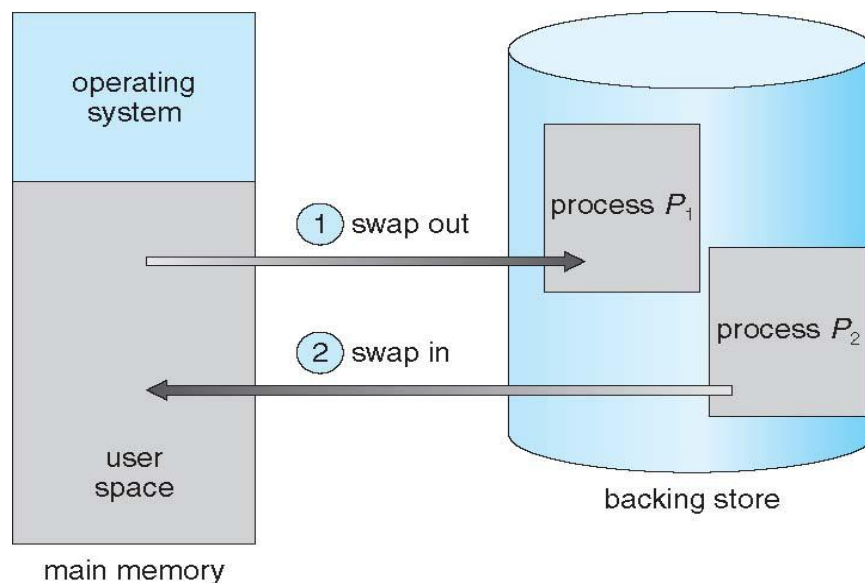
- Translatarea hardware a adreselor (numit și translatarea adreselor)
- Ex. schemă simplă (vom vedea altele la paginare și segmentare)
 - Valoarea din registrul **relocation register** este adăugată la fiecare adresă virtuală
 - Registrul base se numește acum relocation register



Silberschatz, pg. 320

SWAPPING

- Memoria internă poate fi insuficientă pentru păstrarea tuturor proceselor în memorie
 1. Procesele pot fi scoase din memorie
 - Ex. în cazul politicii de planificare bazat pe priorități
 - Procese cu prioritate mai mică sunt interschimbate cu procese mai prioritare
 2. Depozitate temporar pe disc
 - Important să se furnizeze acces direct la imaginile de pe disc
 3. Ulterior readuse în memoria internă pentru continuarea execuției – într-o manieră dinamică



Silberschatz, pg. 322

SWAPPING

- Gestiunea proceselor scoase temporar din memorie
 - **Coadă** de procese swapate
 - Gata de rulare dar așteaptă să fie reîncărcate în memorie
 - Aspecte
 - Trebuie reîncărcat în aceeași locație fizică?
 - Dacă procesul este în stare de așteptare după op. de i/e?
- Swapping-ul este de regulă **dezactivat**
 - Se activează dacă volumul de memorie utilizat depășește o anumită limită
 - Se dezactivează din nou când necesarul de memorie scade sub această limită
- **Timpul necesar** pentru schimbarea de context cu swapping
 - Poate deveni foarte mare
 - Ex. proces de 100MB pe disc cu rata de transfer de 50MB/sec
 - Plus latența de 8 ms
 - Timp de salvare pe disc: 2008 ms
 - Timp de reîncărcare în memorie a urm. proces: 2008 ms
 - Total > 4 sec
 - Soluție: reducerea dimensiunii memoriei interschimbate (swapate)

ALOCAREA MEMORIEI

ALOCAREA MEMORIEI

○ Prima abordare

- Alocare contiguă cu partiționare

• Alocare contiguă

- Fiecare proces este încărcat într-o zonă contiguă de memorie
- Întreg spațiul de adrese este alocat la adrese consecutive
 - Fără locații interpuse

• Partiționare

- Fiecare proces ocupă o partiție

○ Partiționare fixă

- Împărțirea memoriei interne în **partiții de dimensiuni fixe**
 - Egale sau diferite
 - Fiecare partiție poate conține un singur proces

○ Partiționare dinamică

- Împărțirea memoriei interne în **partiții de dimensiuni variabile**

PARTIȚIONARE FIXĂ

○ Partiții de dimensiuni egale

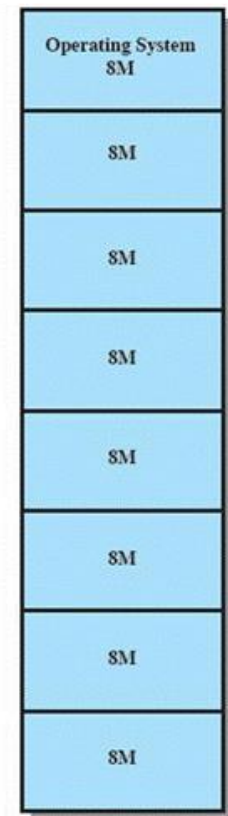
- Procesele de dimensiune mai mică sau egală cu dimensiunea partiției pot fi încărcate într-o partiție liberă

- SO poate interschimba un proces din partiție cu unul de pe disc (swapping)

○ Dezavantaje

- Unele programe pot fi prea mari și nu încap în partiție
 - Soluție – proiectarea programului aî. să fie doar o parte din program în memorie în orice moment
- Utilizarea ineficientă a memoriei interne
 - Fiecare program ocupă o întreagă partiție
 - Nu contează ce dimensiune are
 - **Fragmentare internă**
 - Spații de memorie nefolosite datorită faptului că procesul este mai mic decât partiția

Stallings, pg. 312

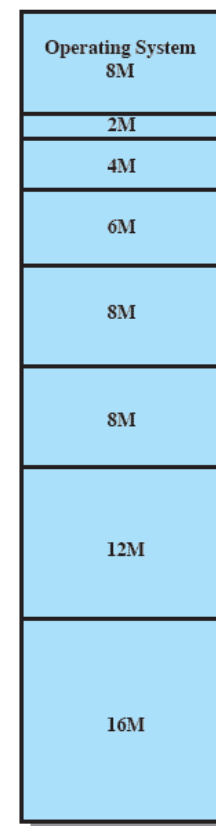


(a) Equal-size partitions

PARTIȚIONARE FIXĂ

○ Partiții de dimensiuni diferite

- Reduc dezavantajele dar nu le elimină
- Dezavantaje
 - Gradul de multiprogramare este limitat de numărul de partiții
 - Unele programe pot fi în continuare prea mari și deci nu pot fi încărcate în nici o partiție
 - **Fragmentare internă** – dar mai redusă



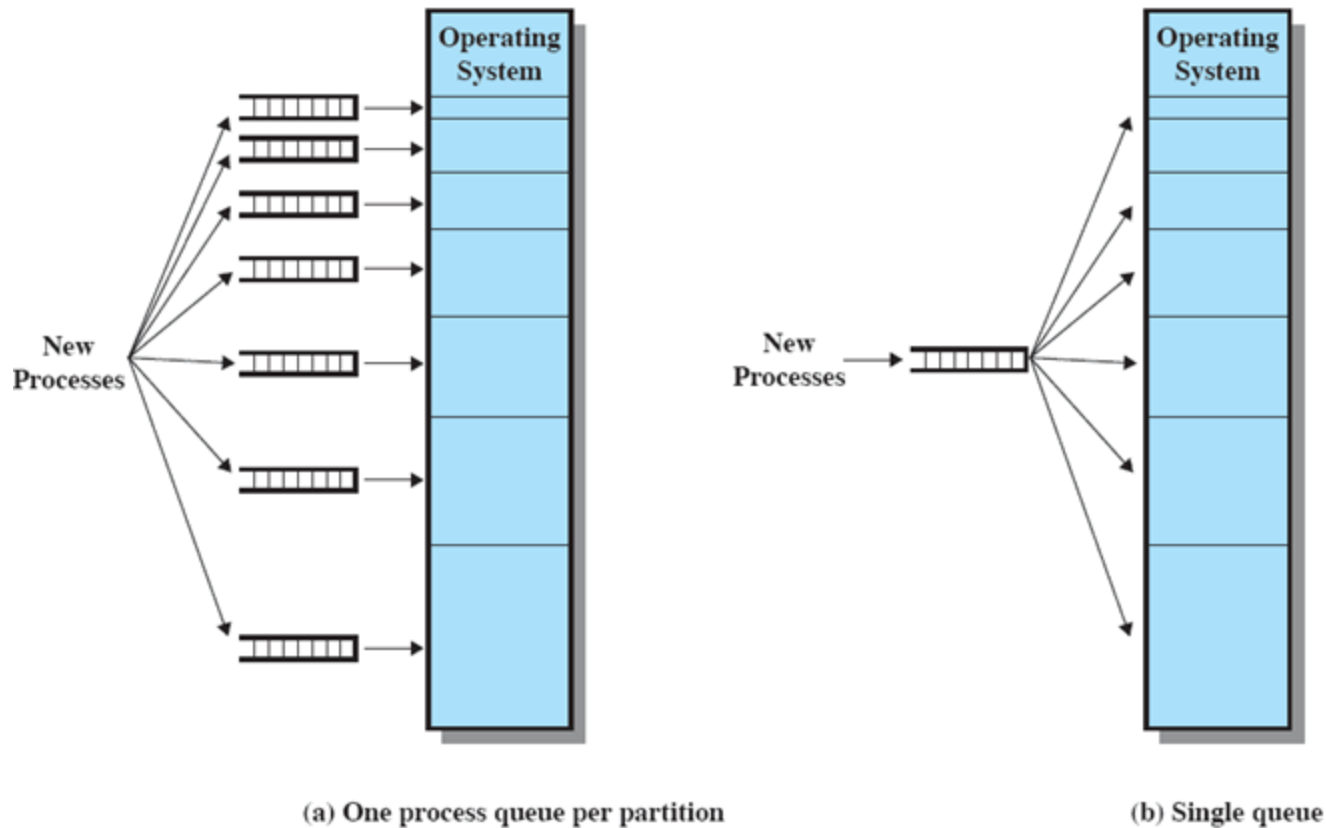
(b) Unequal-size partitions

PARTIȚIONARE FIXĂ

○ Asignarea partițiilor

- În cazul partițiilor fixe de dimensiuni egale – asignare simplă
 - Tabela partițiilor libere
 - Dacă există o intrare liberă, atunci se poate încărca noul proces
 - Toate partițiile de dimensiune egală
 - Nu necesită luarea unei decizii complexe
 - Dacă nu există intrare liberă, atunci un proces este scos temporar și altul încărcat (alegere făcută de planificator)
- În cazul partițiilor fixe de dimensiuni diferite
 - Asignarea procesului la cea mai mică partiție în care încap
 - Este nevoie de **câte o coadă** de planificare pentru fiecare partiție
 - Conține procesele care așteaptă pentru partiția respectivă
 - Avantaj – minimizarea fragmentării interne
 - Dezavantaj – pot exista partiții neutilizate în timp ce la alte partiții coada proceselor este mare (deși ar putea folosi partiția liberă)
 - Folosirea unei **singure cozi** de planificare pentru toate procesele
 - La încărcare se selectează cea mai potrivită (apropiată ca dimensiune) partiție liberă

PARTIȚIONARE FIXĂ – ASIGNAREA PARTIȚIILOR



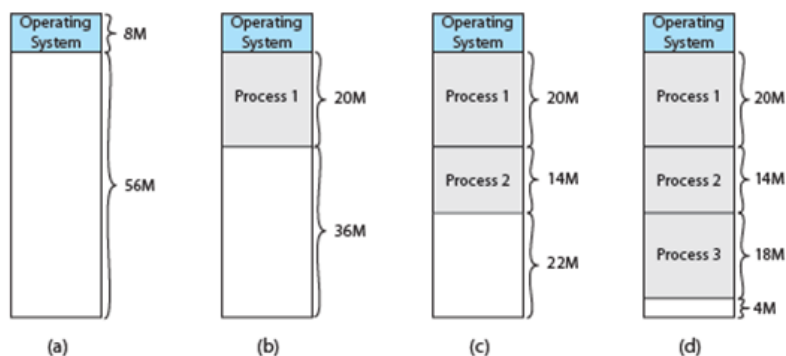
Stallings, pg. 313

Figure 7.3 Memory Assignment for Fixed Partitioning

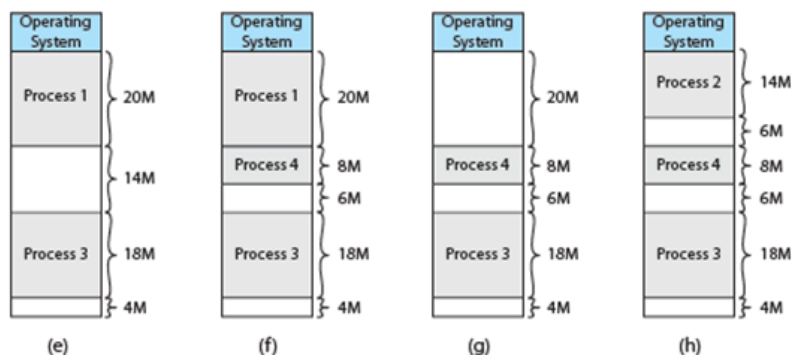
PARTIȚIONARE DINAMICĂ

○ Partiții de dimensiuni variabile

- Număr de partiții variabil
- Inițial tot spațiul utilizator e liber
- Se alocă exact necesarul de memorie pentru procese
- Sistemul menține o tabelă cu partițiile ocupate / libere



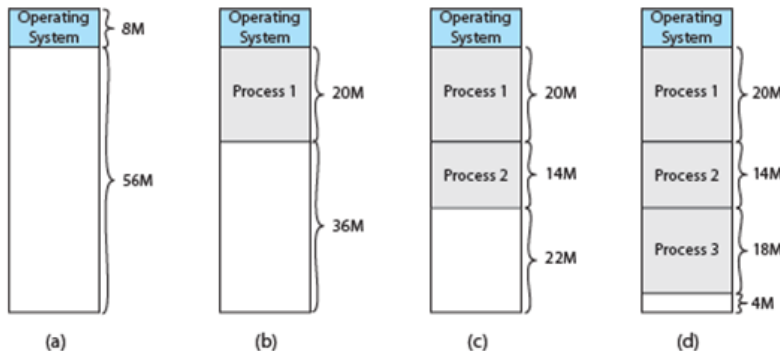
Stallings, pg. 315



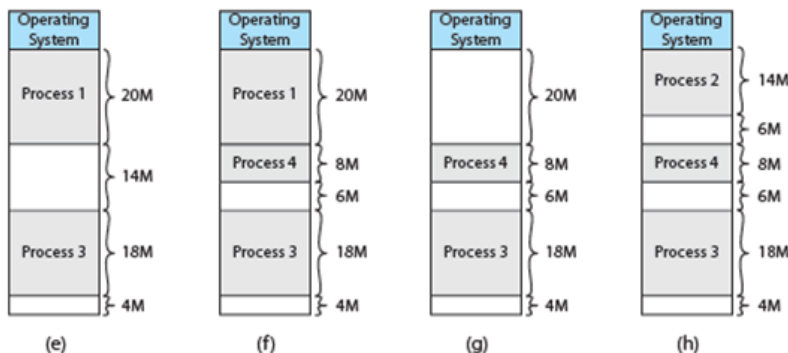
PARTIȚIONARE DINAMICĂ

○ Partiții de dimensiuni variabile

- Număr de partiții variabil
- Inițial tot spațiul utilizator e liber
- Se alocă exact necesarul de memorie pentru procese
- Sistemul menține o tabelă cu partițiile ocupate / libere



Stallings, pg. 315



○ Dezavantaje

○ Fragmentare externă

- Memoria devine din ce în ce mai fragmentată
- Utilizarea devine tot mai ineficientă

○ Soluție:

Compactare

- SO deplasează partițiile aî. să fie contigue
- Consum mare de timp și procesare

PARTIȚIONARE DINAMICĂ

○ Algoritmi de asignare a spațiului liber

First-fit

- Alege primul spațiu liber suficient de mare
- Caută de la începutul tabeli

Best-fit

- Alege spațiu liber cel mai apropiat ca dimensiune
- Caută în întreaga listă

Next-fit

- Alege următorul spațiu liber suficient de mare
- Caută începând de la adresa ultimei asignări

FRAGMENTARE

- Problemă fundamentală a metodelor de alocare
 - **Fragmentară internă**
 - Se risipesc părți din spațiul alocat
 - Memoria alocată este mai mare decât spațiul necesar
 - **Fragmentare externă**
 - Se risipesc părți din spațiul nealocat
 - Se poate reduce prin compactare
 - Doar dacă relocarea este dinamică – în timpul execuției
- Și zona de swapping prezintă aceleași probleme de fragmentare

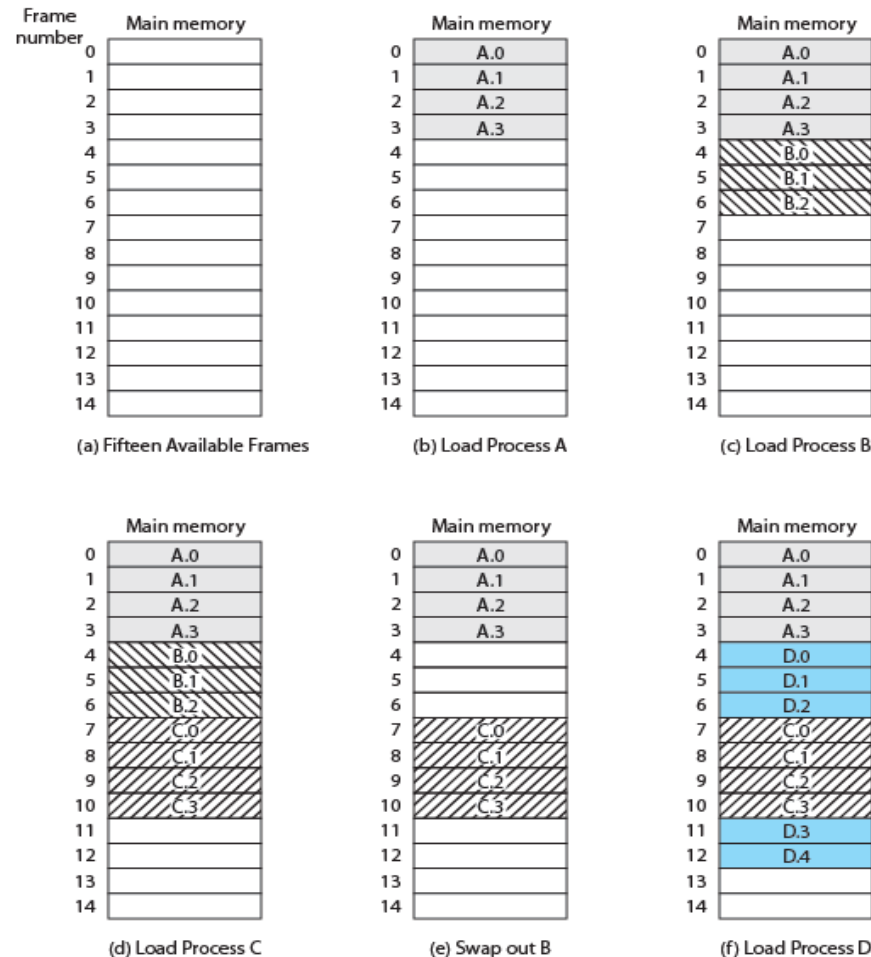
PAGINARE

PAGINARE

- Permite ca spațiul de adrese fizic al procesului să fie **noncontiguu**
 - Pentru a evita fragmentarea externă și nevoia compactării
- Memoria fizică se împarte în **cadre de pagină** (page frame)
 - Blocuri de memorie de dimensiune fixă (relativ mică)
 - Între 512 B și 16 MB (putere al lui 2)
- Memoria logică se împarte în **pagini**
 - Aceeași dimensiune ca și cadrele de pagină

PAGINARE

○ Asignarea proceselor la cadre de pagină



Stallings, pg. 322

Figure 7.9 Assignment of Process Pages to Free Frames

PAGINARE

- Trebuie ținută evidența cadrelor de pagină libere
 - Pentru a rula un program de dimensiunea N pagini, trebuie găsite N cadre libere în memoria fizică unde se va încărca programul
 - Paginile pot fi dispersate – nu neaparat la adrese consecutive
 - Alocare necontiguă
- Tabela de pagini
 - Conține maparea adreselor virtuale la adrese fizice
 - Fiecare proces are o tabelă de pagini – gestionată de SO
- Rămâne problema (deși mult mai redusă a) fragmentării interne
 - Ultima pagină poate fi doar parțial utilizată

PAGINARE

○ Tabela de pagini pentru exemplul precedent

○ Momentul (f)

0	0	0	—
1	1	1	—
2	2	2	—
3	3		

Process A
page table

Process B
page table

0	7	0	4
1	8	1	5
2	9	2	6
3	10	3	11
		4	12

Process C
page table

Process D
page table

13
14

Free frame
list

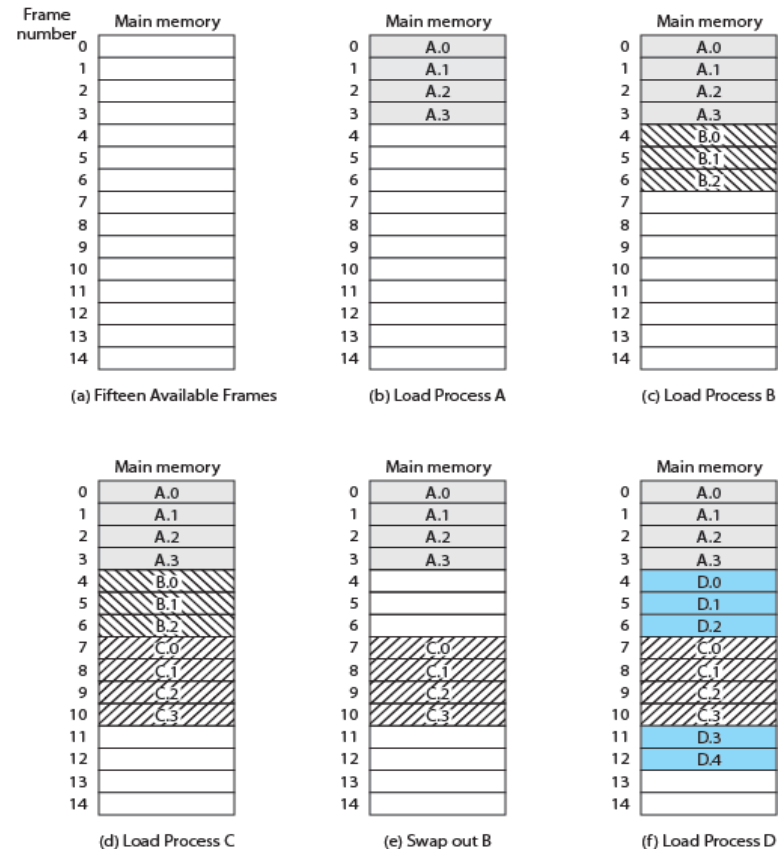


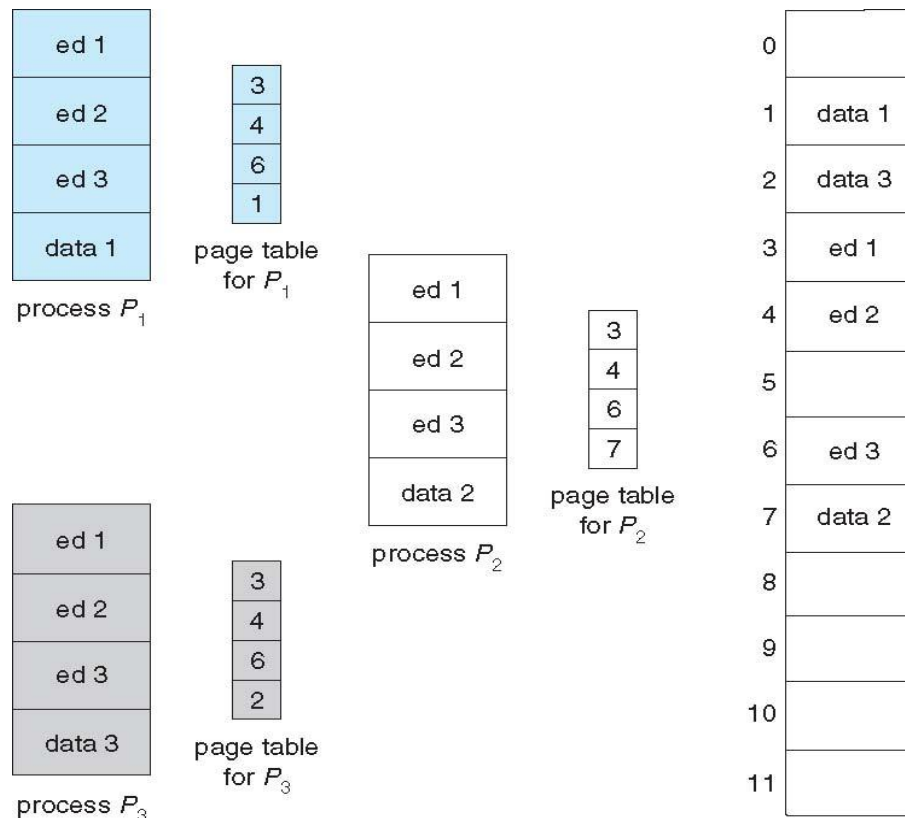
Figure 7.9 Assignment of Process Pages to Free Frames

PAGINI PARTAJATE

Memorie partajată

- Utilitate

- Cod - mai multe procese, același program
- Comunicare între procese prin memorie partajată

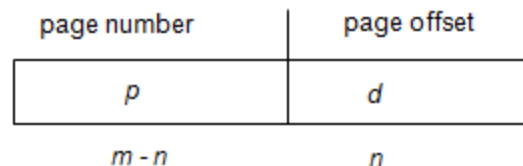


Silberschatz, pg. 337

PAGINARE

○ Tabela de pagini

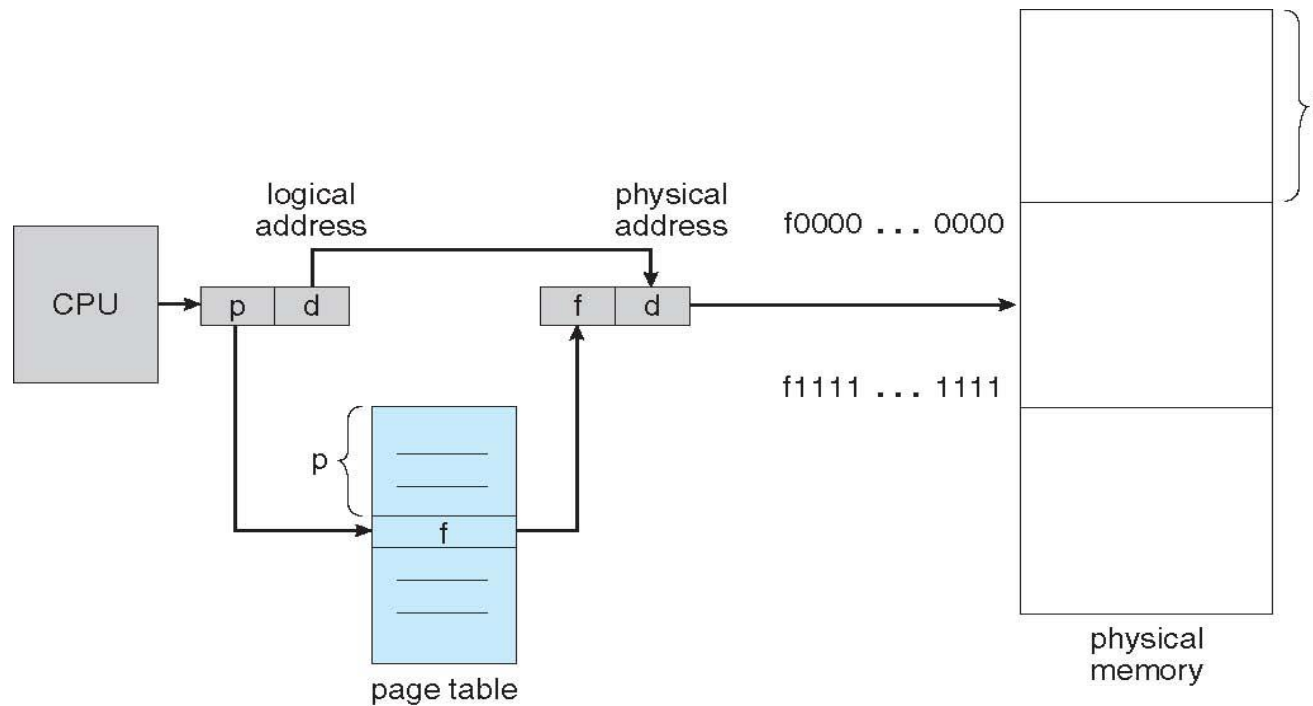
- Conversia adreselor logice în adrese fizice
 - Adresele virtuale (generate de procesor) sunt împărțite în
 - Număr de pagină
 - index în tabela de pagini
 - Tabela de pagini conține adresele de început a fiecărei pagini în memoria fizică
 - Deplasament (offset)
 - Poziția relativă față de adresa de început a paginii



Pentru un spațiu de adrese de dimensiunea 2^m și dimensiunea de pagină: 2^n

PAGINARE

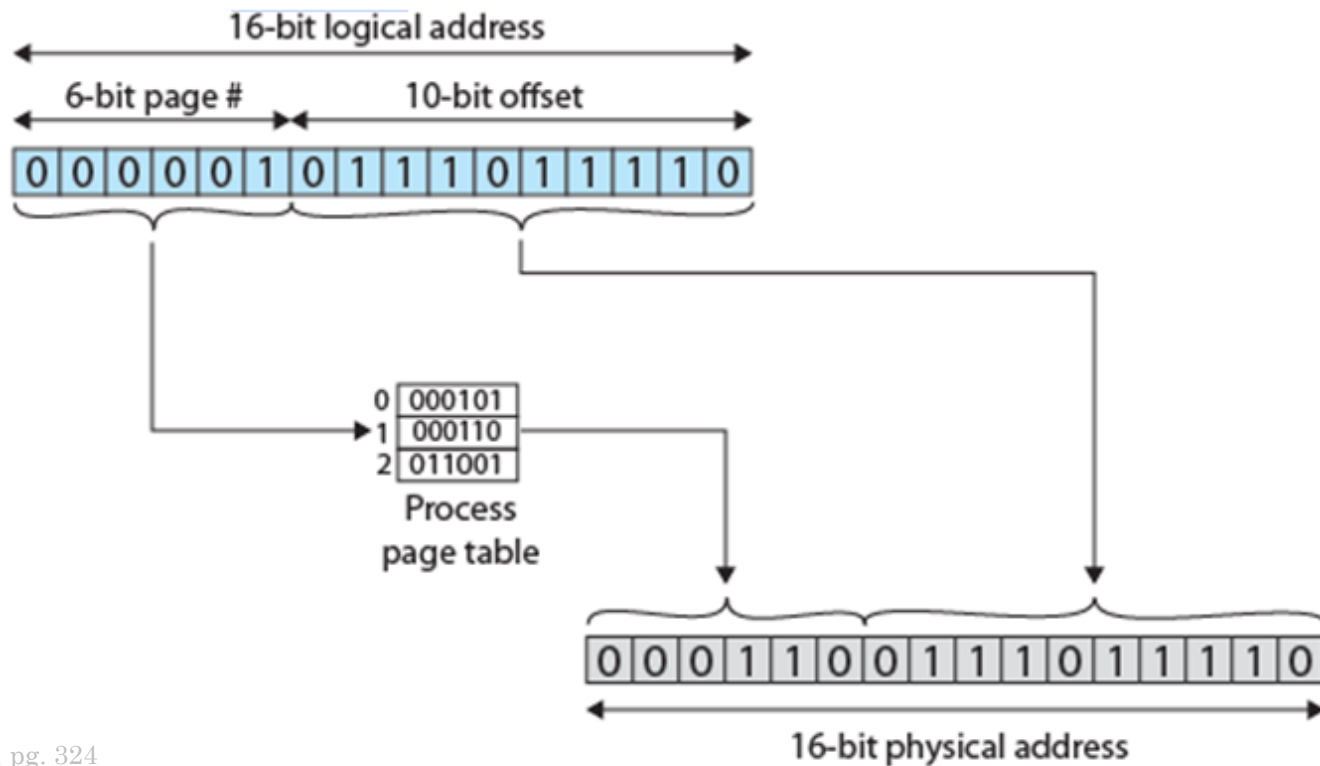
- Modul de efectuare a conversiei de la adresă logică la adresă fizică
 - Suport hardware



PAGINARE

○ Exemplu

- dimensiunea paginii: 1 KB



Stallings, pg. 324

(a) Paging

EXERCİȚIU

○ Translatarea adreselor

○ Considerăm

- Adrese virtuale pe 8 biți
- Adrese fizice pe 10 biți
- Dimensiunea unei pagini de 64 bytes

○ Se cere

- Numărul de pagini virtuale: **4**
- Numărul de pagini fizice: **16**
- Numărul de intrări în tabela de pagini: **4**
- Dacă tabela de pagini = [2, 5, 1, 8] adresa virtuală 241 se va traduce în adresa fizică:

1	1	1	1	0	0	0	1
---	---	---	---	---	---	---	---

1	0	0	0	1	1	0	0	0	1
---	---	---	---	---	---	---	---	---	---

EXERCİȚIU

○ Translatarea adreselor

- Cum am putea generaliza pentru adrese virtuale pe m biți, adrese fizice pe n biți și offsetul de pagină de k biți?
 - Numărul de pagini virtuale:
 - Numărul de pagini fizice:
 - Numărul de intrări în tabela de pagini:
 - Dimensiunea memoriei virtuale:
 - Dimensiunea memoriei fizice:
 - Dimensiunea tabelului de pagini:

EXERCİȚIU

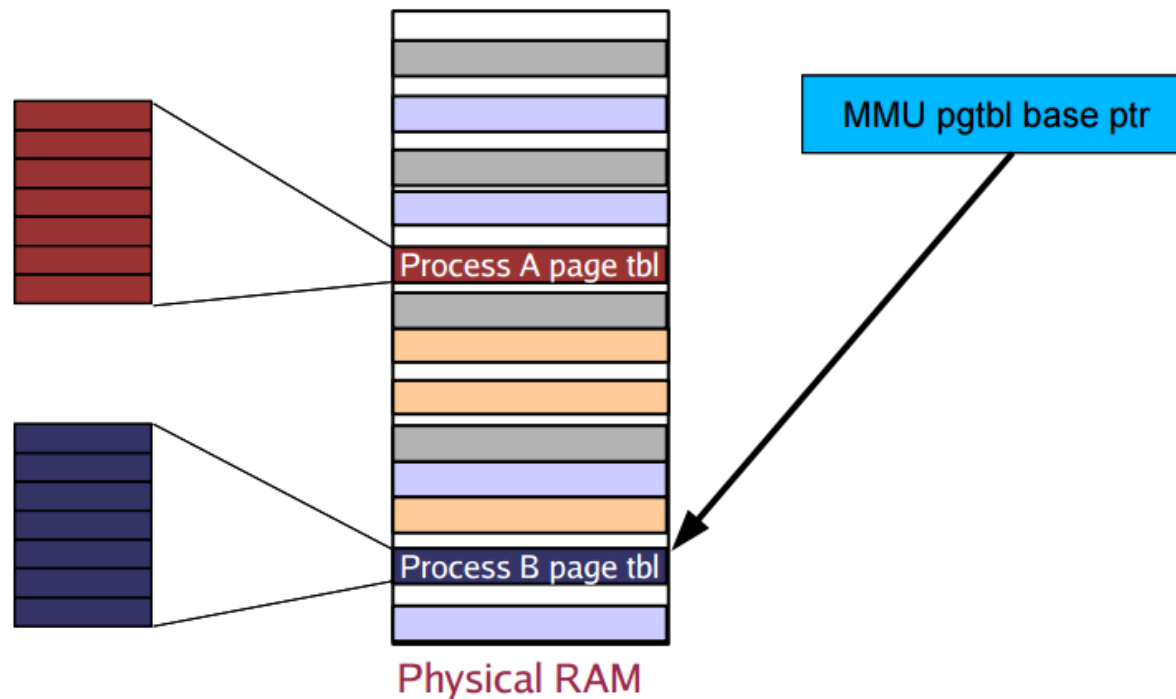
○ Translatarea adreselor

- Cum am putea generaliza pentru adrese virtuale pe m biți, adrese fizice pe n biți și offsetul de pagină de k biți?
 - Numărul de pagini virtuale: 2^{m-k}
 - Numărul de pagini fizice: 2^{n-k}
 - Numărul de intrări în tabela de pagini: 2^{m-k}
 - Dimensiunea memoriei virtuale: 2^m
 - Dimensiunea memoriei fizice: 2^n
 - Dimensiunea tabelului de pagini: 2^{m-k} x dimensiunea unei intrari
 - O intrare include cel puțin $n-k$ biti pt numărul de cadru de pagină
 - + alti biti de control (vedem in curand)

PAGINARE

○ Tabele de pagini

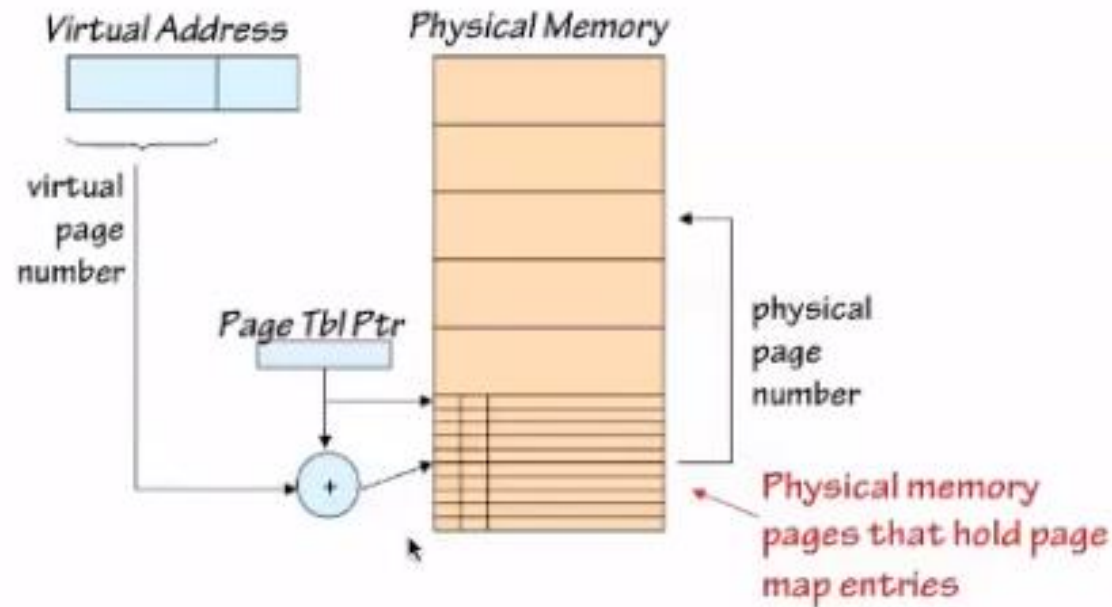
- Stocate în memorie
- MMU accesează tabela corespunzătoare procesului prin registrul **page table base register**
 - Indică adresa fizică unde începe tabela de pagini a procesului



PAGINARE

- Problemă

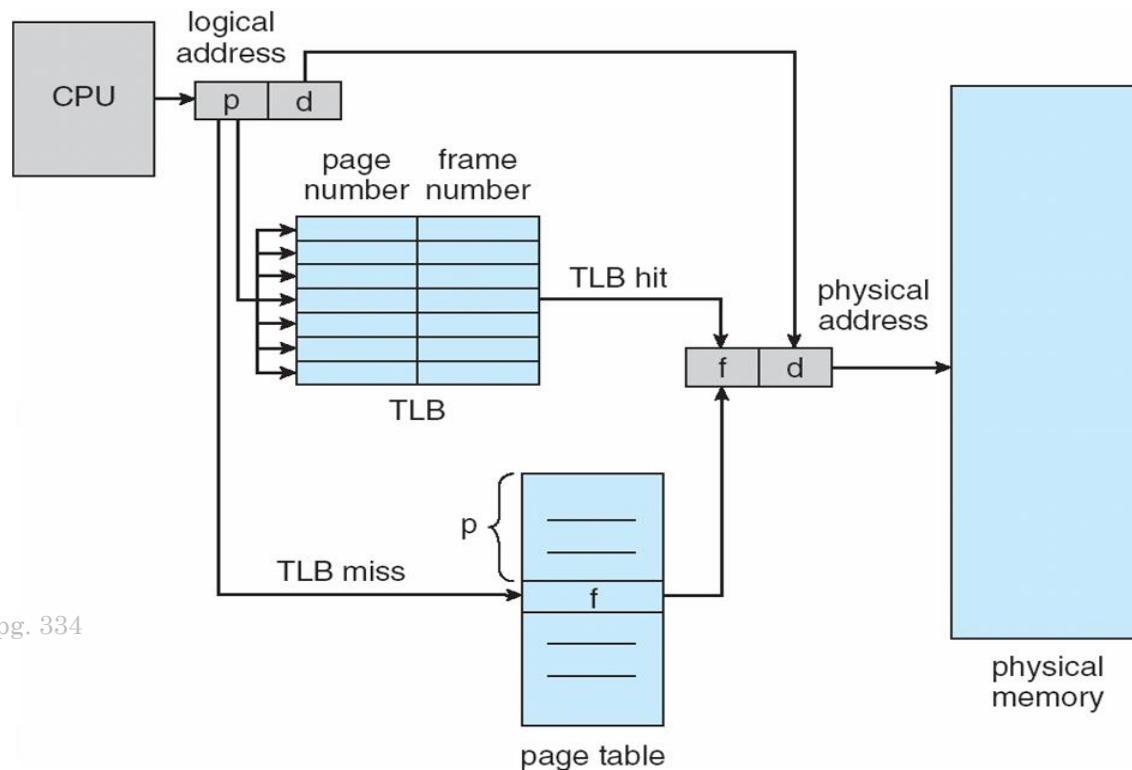
- Fiecare acces la date / instrucțiuni necesită două accesări ale memoriei
 - Pt tabela de pagini și pt data/instrucțiune



PAGINARE CU TLB

○ Soluție

- **Translation look-aside buffer (TLB)** - memorie asociativă
 - Cache hardware de căutare rapidă (Content Addressable Memory)
 - Dimensiuni mici – 64 – 1024 de intrări
 - **Cache al tabelului de pagini**
 - Reține mapările recente (nr pagină – nr cadru de pagină)



Stallings, pg. 334

TLB – MEMORIE ASOCIATIVĂ

- Memoria asociativă permite căutare paralelă
 - regăsire mult mai rapidă
 - TLB
 - Asocieri de forma: **nr pagină (p), cadru de pagină (f)**
 - Fiecare intrare conține și identificatori de spațiu de adrese
 - ASID – address-space identifiers
 - Indetifică procesele care dețin spațiul de adrese pt a furniza protecție
 - Translația adresei
 - Dacă **p** este regăsit în memoria asociativă (TLB), returnează **f**
 - TLB hit – s-a găsit potrivire
 - Altfel accesează tabela de pagini pentru a-l regăsi pe **f**
 - TLB miss – nu s-a găsit potrivire
 - -> trebuie căutat în tabela de pagini
- Încetinește translația adreselor

TIMP DE ACCESARE EFECTIV

- Căutarea asociativă (lookup în TLB)
 - Necesită ε unități de timp
- Timpul unui ciclu de memorie
 - Presupunem că este 1 microsecundă
- Rata de găsim a unei potriviri (hit-ratio)
 - Procent din timp când numărul de pagină este regăsit în memoria asociativă (TLB hit)
 - Depinde de numărul de regiștri din care este formată memoria asociativă
 - Notăm cu α
- **EAT** = $\text{HitRatio} * (\text{TLBaccessTime} + \text{MemAccessTime}) + (1 - \text{HitRatio}) * (\text{TLBaccessTime} + \text{PTaccessTime} + \text{MemAccessTime})$
- Timpul de acces efectiv (Effective Access Time)
$$\text{EAT} = (1 + \varepsilon)\alpha + (2 + \varepsilon)(1 - \alpha) = 2 - \alpha + \varepsilon$$

- Folosim AMAT în contextul memoriei cache
- Folosim EAT în contextul paginării

TIMP DE ACCESARE EFECTIV

○ Exercițiu

- Căutarea asociativă (lookup în TLB) : $\epsilon = 5 \text{ ns}$
- Timpul unui ciclu de memorie = 100 ns
- Rata de găsim a unei potriviri (hit-ratio) : α

$$\text{EAT} = (100 + 5)\alpha + (100 + 100 + 5)(1 - \alpha)$$

○ Dacă $\alpha = 80\%$

$$\text{EAT} = (100 + 5)*0.8 + (100 + 100 + 5)*0.2 = 125 \text{ ns}$$

○ Dacă $\alpha = 98\%$

$$\text{EAT} = (100 + 5)*0.98 + (100 + 100 + 5)*0.02 = 107 \text{ ns}$$

DIMENSIUNEA TABELEI DE PAGINI

○ Considerăm

- Spațiu de adrese pe 32 biți (4 GB)
 - Pagini de 4 KB
 - O intrare în tabela de pagini 4 octeți
- Prin urmare tabela de pagini are dimensiunea de 4 MB per proces!
- Observații
 - De cele mai multe ori spațiul de adrese este rar
 - Nu se folosesc totii cei 2^{32} octeți

○ Soluții

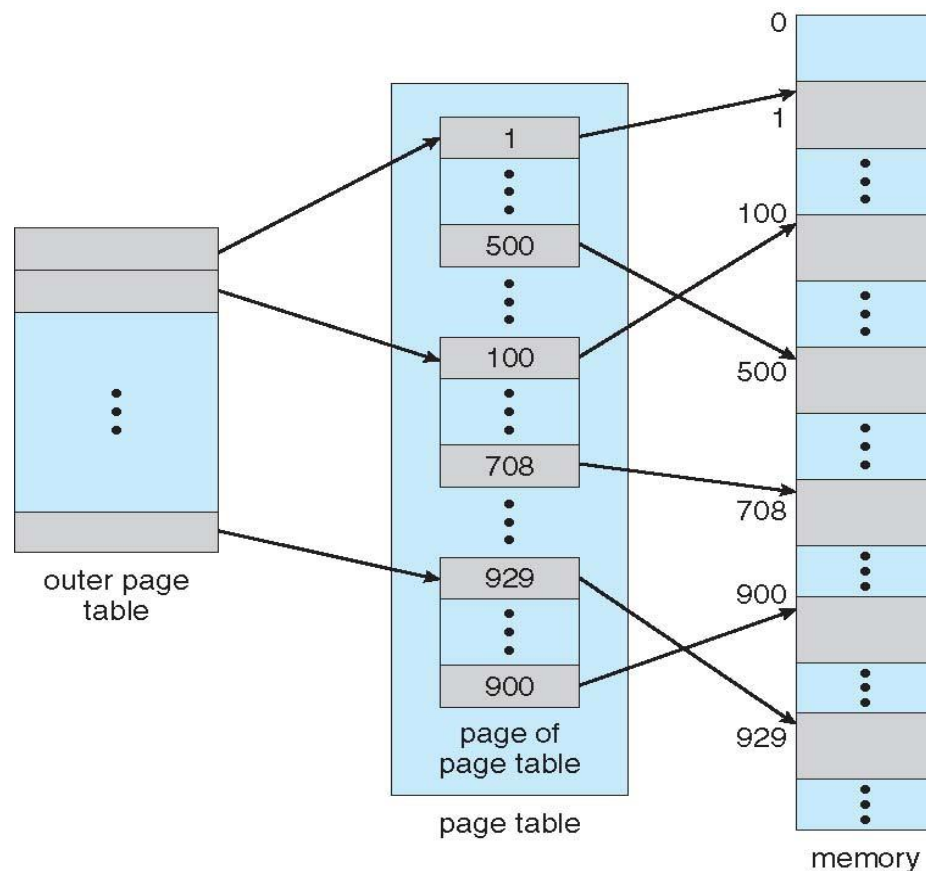
- Schimbarea structurii de implementare a tabelii de pagini pentru a economisi memorie
- Compromis între timpul de translatare și dimensiunea tabelii de pagini

STRUCTURA TABELEI DE PAGINĂ

- Structurile de gestiune pot deveni foarte mari
 - Soluție
 - Paginare ierarhică
 - Tabele de pagini cu dispersie
 - Tabele de pagini inversate

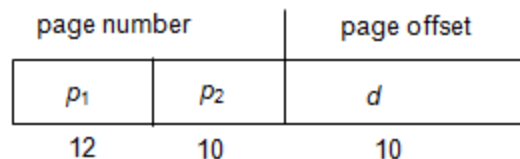
PAGINARE IERARHICĂ

- Spațiul de adrese logic este divizat în mai multe tabele de pagini
- Tabelă de pagini cu două niveluri
 - Metodă simplă
 - Pagină tabelă de pagini



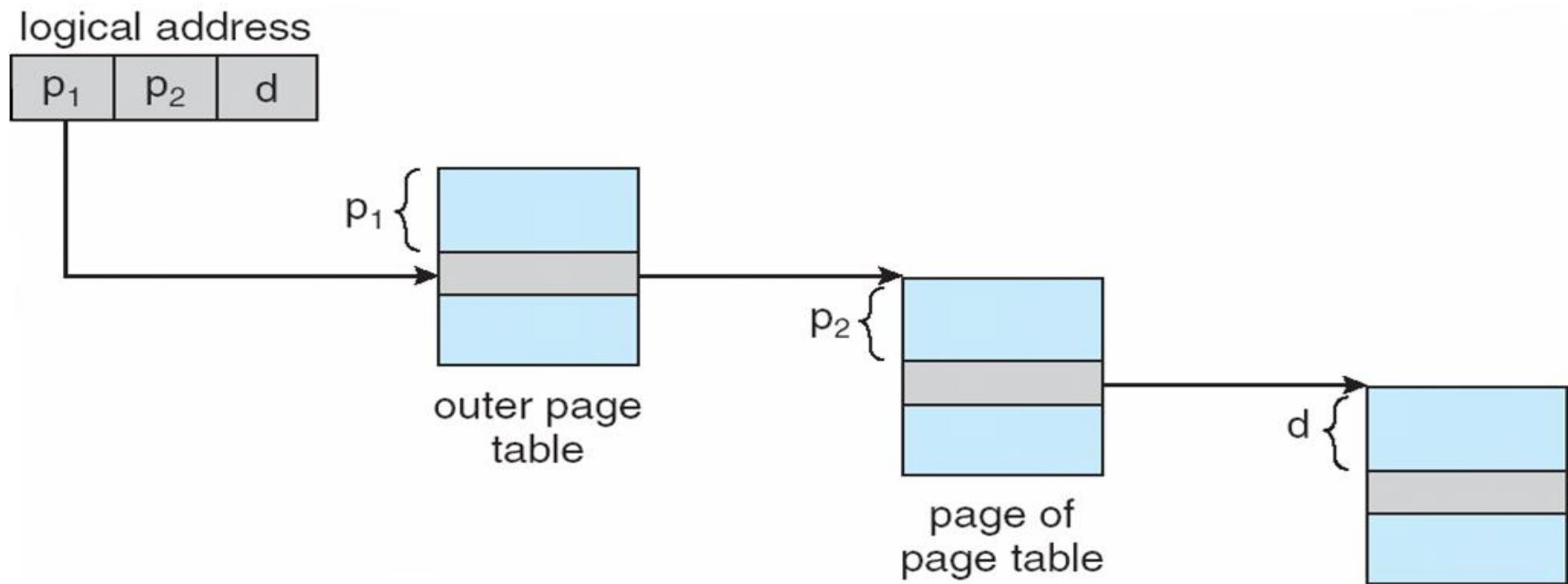
PAGINARE IERARHICĂ - EXEMPLU

- Spațiu de adrese pe 32-biți cu pagini de dimensiune 1KB
- Adresa logică
 - Numărul de pagină – 22 biți
 - Offset de pagină – 10 biți (notăm cu d)
- Tabela de pagini este paginată
 - Numărul de pagină se împarte în
 - Număr de pagină exterioară – 12 biți (notăm cu p_1 – index în tabela de pagini exterioară)
 - Număr de pagină interioară – 10 biți (notăm cu p_2 – index în tabela de pagini interioară)



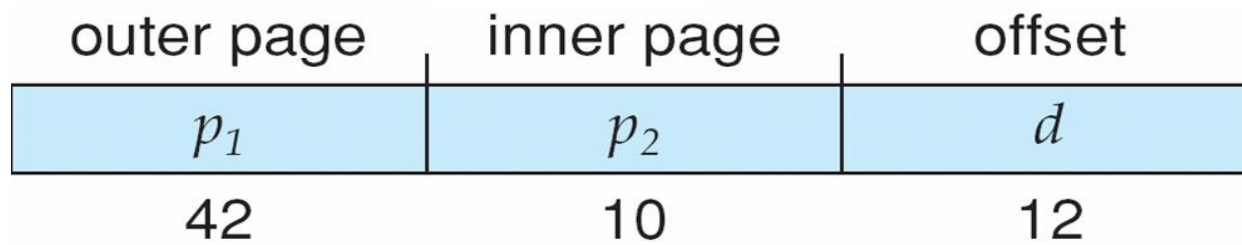
PAGINARE IERARHICĂ

- Translația adresei – exemplu pt două niveluri de paginare

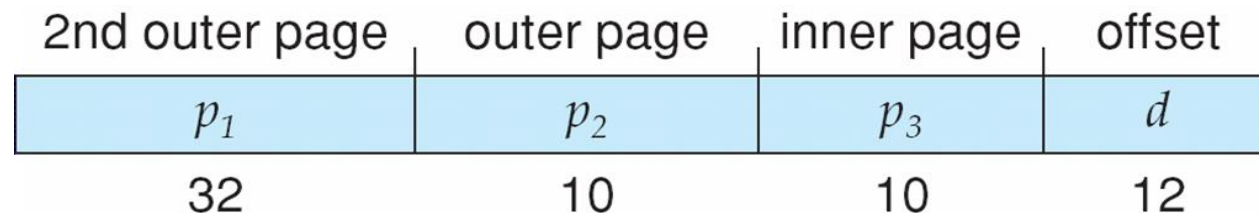


PAGINARE IERARHICĂ

- Când ierarhia pe două niveluri nu mai este suficientă



Silberschatz, pg. 339



TIMP DE ACCESARE EFECTIV

○ Exercițiu

- Tabela de pagini ierarhică cu 3 niveluri
- Căutarea asociativă (lookup în TLB) : $\varepsilon = 5$ ns
- Timpul unui ciclu de memorie = 100 ns
- Rata de găsire a unei potriviri (hit-ratio) : α

$$EAT = (100 + 5)\alpha + (100 + 300 + 5)(1 - \alpha)$$

○ Dacă $\alpha = 80\%$

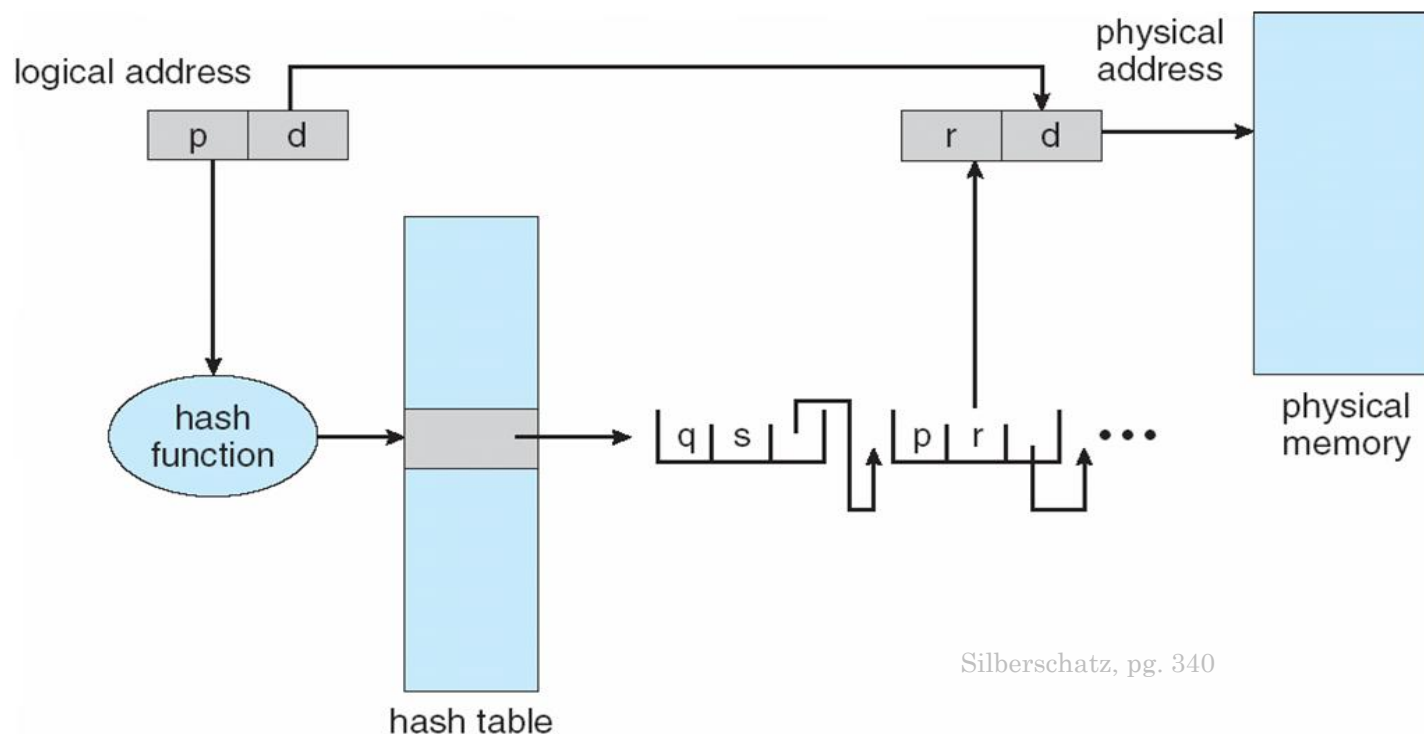
$$EAT = (100 + 5)*0.8 + (100 + 300 + 5)*0.2 = 165 \text{ ns}$$

○ Dacă $\alpha = 98\%$

$$EAT = (100 + 5)*0.98 + (100 + 300 + 5)*0.02 = 111 \text{ ns}$$

TABELE DE PAGINĂ CU DISPERSIE

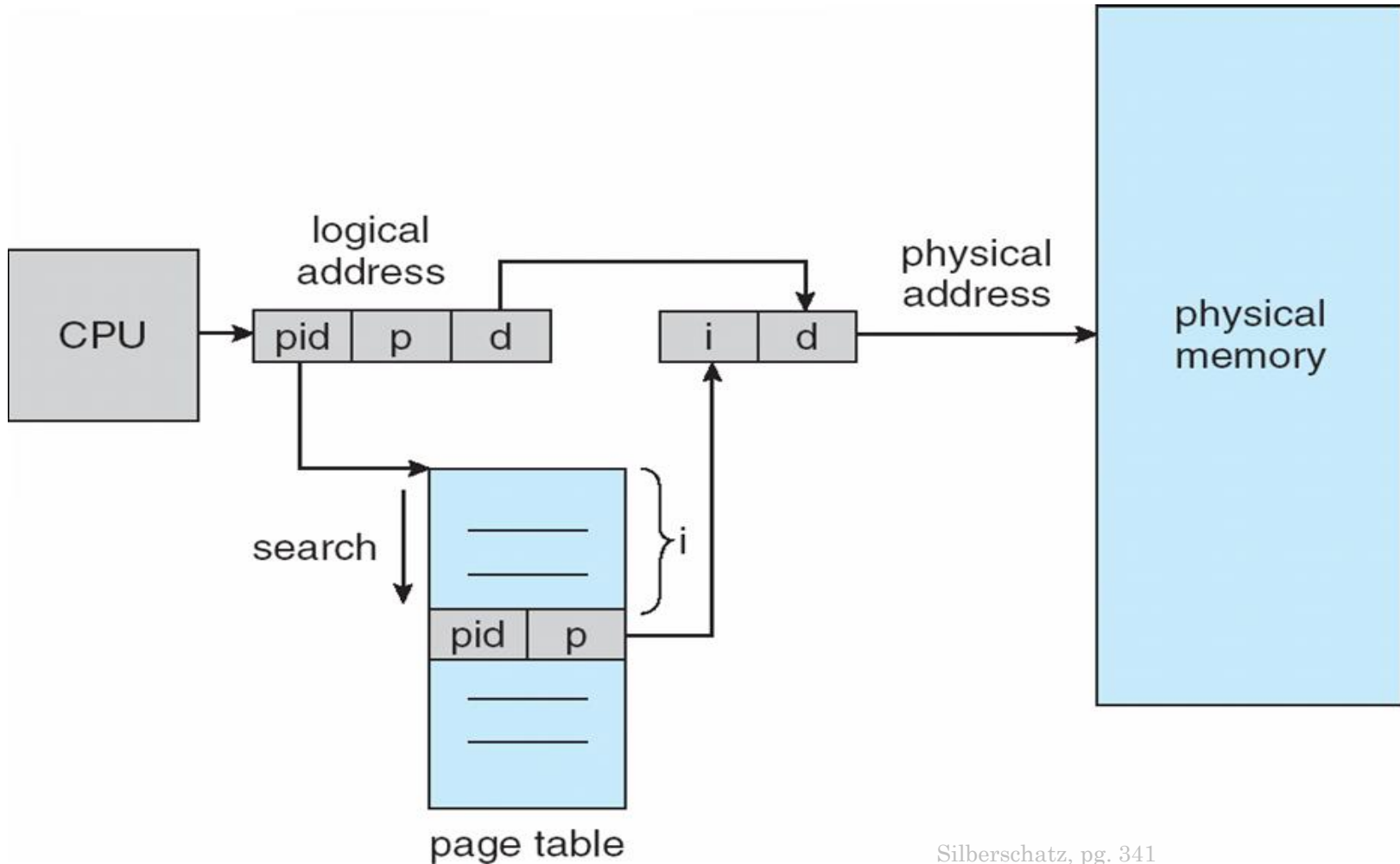
- De regulă la spații de adresă pe > 32 biți
- Tabela de pagini este implementată ca o tabelă de dispersie
 - Intrările conțin lista înlănțuită a asocierilor (pagină - cadru) care se mapează la același cod de dispersie
 - Pe baza numărului de pagină se caută numărul de cadru asociat



TABELE DE PAGINĂ INVERSATE

- Până acum am avut
 - Tabele de pagini pentru fiecare proces
 - Evidența tuturor paginilor logice
- **Abordare inversată** – se ține evidența tuturor cadrelor de pagină din memoria fizică
 - Tabela cadrelor
 - Câte o intrare pentru fiecare cadru de pagină din memoria fizică
 - Informații necesare
 - Adresa virtuală a paginii stocate
 - Informații legate de procesul care deține aceea pagină
- Reduce necesarul de memorie pentru gestiunea tabelelor de pagină
- Crește timpul necesar căutării în tabelă
 - Se pot folosi tabele de dispersie sau TLB-uri
 - Dar cum se poate implementa **partajarea** unui cadru de pagină de mai multe procese? – foarte dificil...

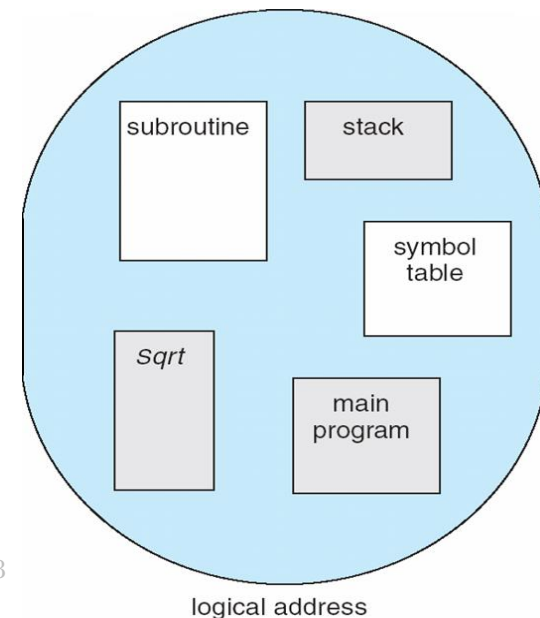
TABELE DE PAGINĂ INVERSATE



SEGMENTARE

SEGMENTARE

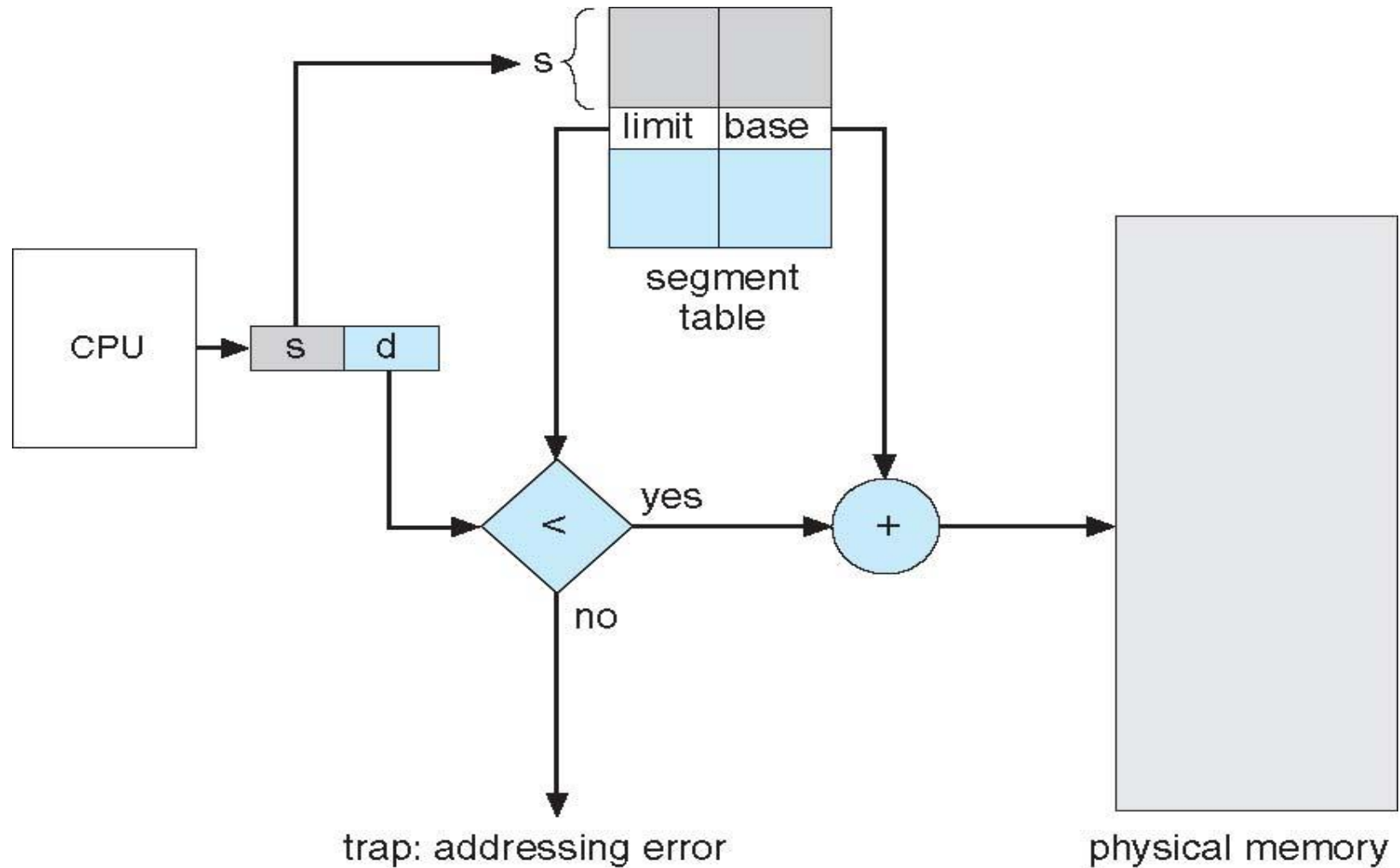
- Metodă de gestiune a memoriei care oferă suport pentru **viziunea utilizatorului** asupra memoriei
 - Cei mai mulți utilizatori nu văd memoria ca un șir lung de octeți ci ca și o **colecție de segmente** cu anumite funcționalități și de dimensiuni variate
- Programul divizat în segmente
 - De dimensiuni variabile, cu o limită maximă
- Segment
 - Unitate logică
 - Exemple
 - Segment de date, segment de cod, etc.
 - Program principal, procedură, funcție, etc.
 - Metodă, obiect, etc.



SEGMENTARE

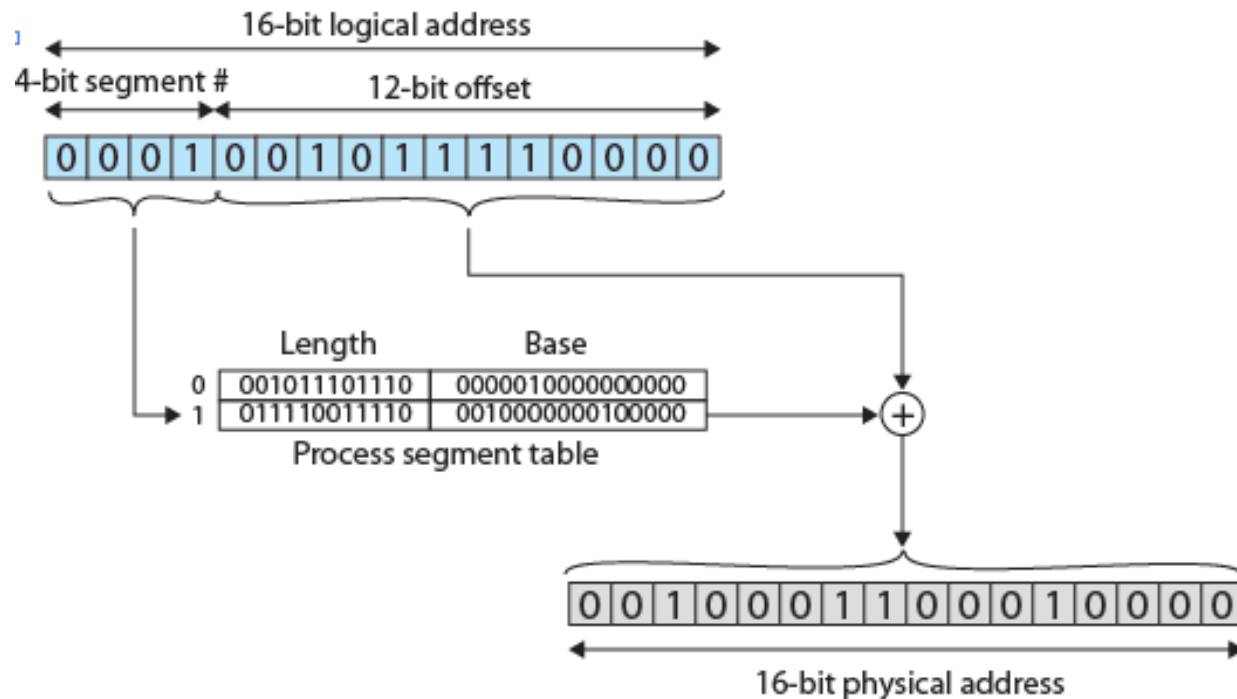
- Generalizarea relocării dinamice
 - În loc de o singură pereche **base** și **limit**, MMU folosește mai multe perechi
 - Seamănă (într-un fel) cu partiționarea dinamică
- Tabela de segmente
 - Fiecare intrare are
 - Base – adresa locației fizice unde începe segmentul
 - Limit – dimensiunea segmentului
- Segment-table base register (STBR)
 - Indică locația tabeli de segmente în memorie
- Segment-table length register (STLR)
 - Indică numărul de segmente folosite de program
 - Numărul de segmente s este valid dacă $s < \text{STLR}$
- Adresa logică
 - Număr de segment
 - Deplasament
- Referențierea unei adrese din afara limitelor
 - Segmentation fault
- Elimină fragmentarea internă
 - Rămâne problema fragmentării externe

SEGMENTARE



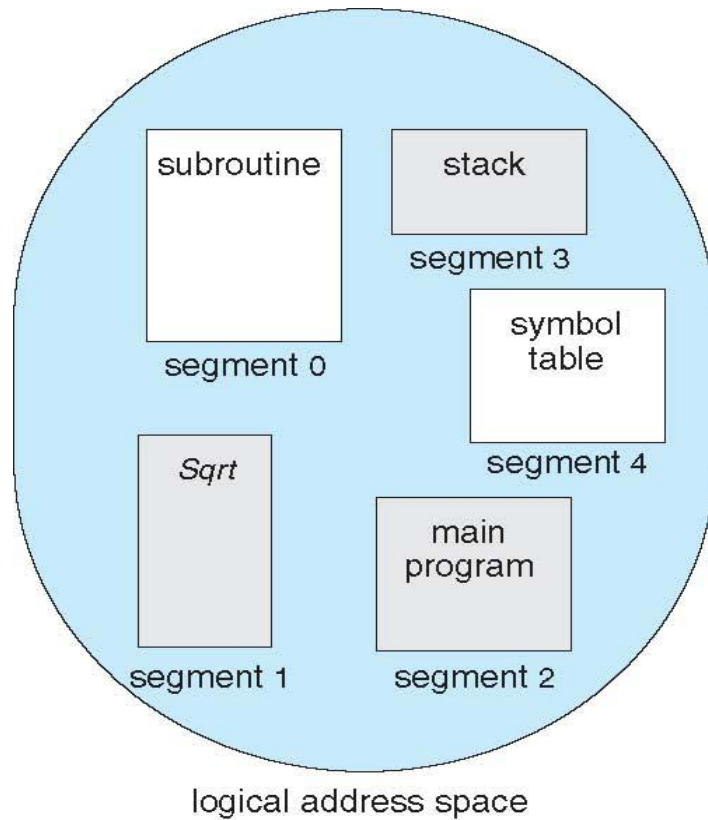
SEGMENTARE

- Modul de efectuare a conversiei de la adresă logică la adresă fizică
 - Suport hardware
- Exemplu



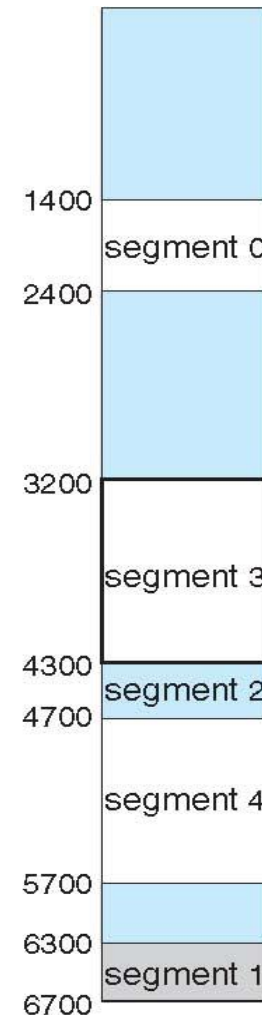
(b) Segmentation

SEGMENTARE - EXEMPLU



	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

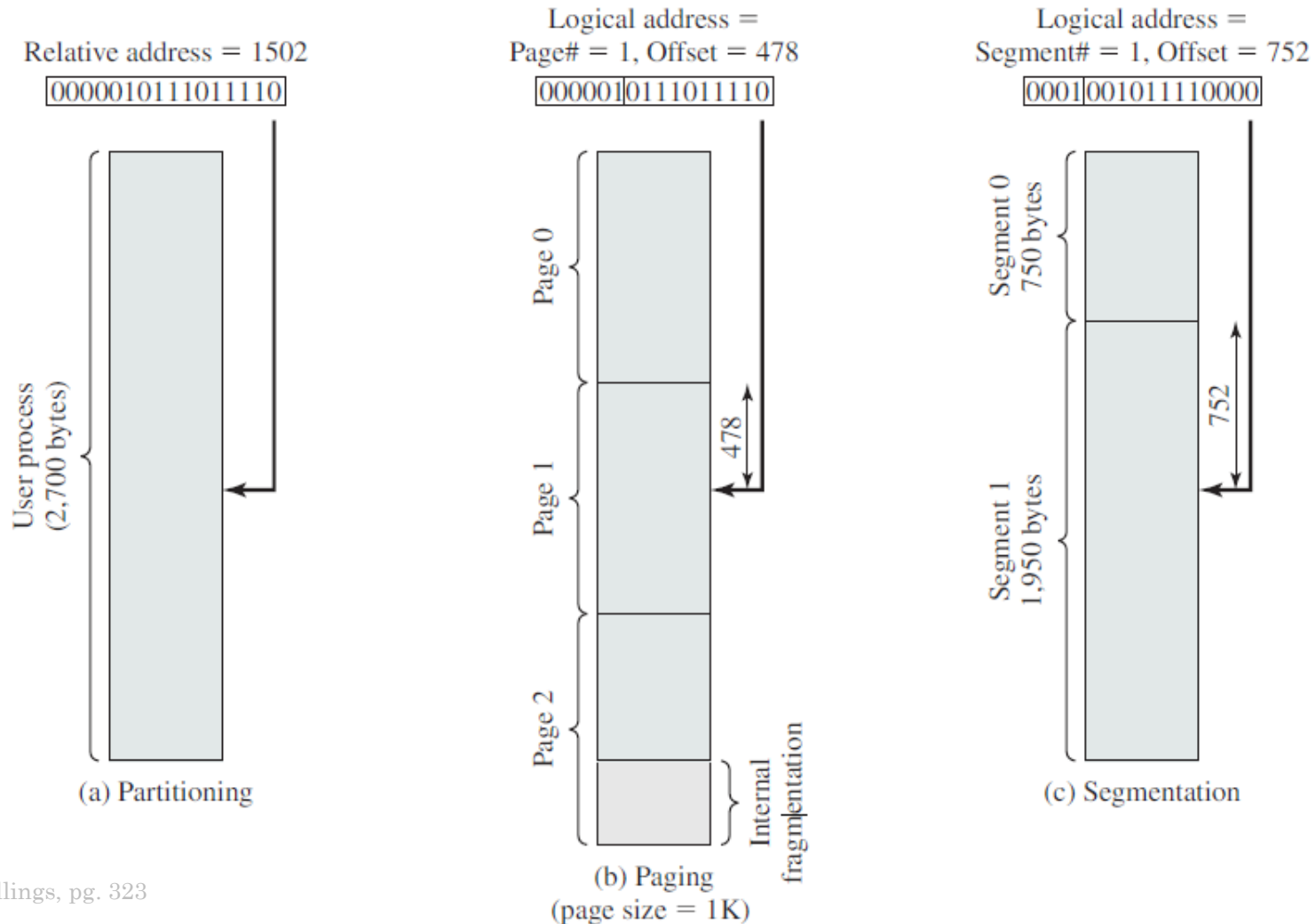
segment table



physical memory

ADRESE VIRTUALE

○ Partitionare vs. Pagine vs. Segmentare



CONCLUZII

- Memoria este o resursă care trebuie multiplexată
- Metode variate
 - Partiționare
 - Paginare
 - Segmentare
- Probleme
 - Fragmentare
 - Internă
 - Externă